A FUZZY ENABLED EDGE NETWORK DEVICE SIMULATION

SOFTWARE APPLICATION

A Master's Project

By

Eric W. Yocam

Fall 2004

APPROVED BY THE DEAN OF THE SCHOOL OF

GRADUATE, INTERNATIONAL, AND SPONSORED PROGRAMS:

_____

Robert M. Jackson, Ph.D., Dean

APPROVED BY THE GRADUATE ADVISORY COMMITTEE:

_____ _____

Ralph Hilzer, Ph.D.

Graduate Coordinator

Seung-Bae Im, Ph.D.

_____

Benjoe Juliano, Ph.D.

PUBLICATION RIGHTS

No portion of this project may be reprinted or reproduced in any manner unacceptable to the usual copyright restrictions without the written permissions of the author.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

ABSTRACT

The objective of this master's project is to build a simulation model represented through the creation of an interactive software application. The simulation model demonstrates that a conventional network edge routing device can be enhanced to produce a high degree of performance achieved with the application of fuzzy logic in contrast to the device performance achieved that does not have fuzzy logic applied to it. The simulation model demonstrates two different (yet simplified) scenarios where one scenario exhibits routing of network traffic found in a conventional network edge routing device (also called an intelligent edge device) and the other scenario exhibits routing of network traffic but with an enhancement made to it that incorporates a number of fuzzy logic principles to the priority control mechanism of the intelligent edge device.

The resulting contrast between these two scenarios demonstrates that by incorporating fuzzy logic principles in a conventional network edge routing device, the result is a much higher degree of performance obtained with no fuzzy logic implementation. Even though the simulation model just emulates two very simplified and conceptual scenarios, the software application enables the user to experiment by making changes in the control parameters and displays how these changes affect the overall simulation model.

The simulation model demonstrates that an intelligent edge device that incorporates fuzzy logic not only blurs the network's edge toward ubiquitous network topology but also enables enhanced support of complex protocols in order to deliver a new breed of services to the consumer. Further, the enhanced support is developed by combining fuzzy logic, the principle of Quality of Service (QoS), and an enhanced version of Multi-protocol Label Switching (MPLS) can be used in combination to address many of the challenges facing today's networks.

INTRODUCTION TO THE PROJECT

Today's network device has not reached its maximum potential in terms of ensuring Quality of Service (QoS) for network services delivered to network users—or, from the Internet service provider perspective, to subscribers.

Current TCP/IP congestion control algorithms cannot efficiently support new and emerging services needed by the Internet community. In fact, one could argue that network congestion control, still remains a critical and high priority issue especially given the growing size, demand, and speed (bandwidth) of the network [17].

PURPOSE OF THE PROJECT

The purpose of this project is to build a simulation model represented through the creation of a software application used by a software user (See Figure 1). The simulation model demonstrates that a conventional network edge routing device can be enhanced to produce a high degree of performance with the application of fuzzy logic in contrast to the device performance that does not have fuzzy logic applied to it.

SCOPE (DESCRIPTION) OF THE PROJECT

By combining MPLS protocol, fuzzy logic and an edge network device, I purport that network traffic can be prioritized between high and low priority traffic and then routed more effectively thereby increasing the total throughput at the edge of the network while maintaining a QoS goal for high traffic demand. This will be shown through the development of simulator software where packets will be randomly prioritized, packets will be transferred along a pre-configured path and at each stop the packet will be compared to a device policy. Each device policy will control the acceptance or rejection of the packet based on whether fuzzy logic control is enabled or not enabled. The fuzzy logic control is based on a fuzzy logic class library

developed by a company called Louder than a Bomb (http://ffll.sourceforge.net). The simulator provides output after each trial is ran by the user. This output is divided between results from enabling fuzzy logic and disabling it and then compared to support the purported conclusion of this paper that in combination with MPLS and an edge network device using fuzzy logic translates into a slight increase in throughput or network performance over not using fuzzy logic.

The simulation model demonstrates two different (but simplified) scenarios where one scenario exhibits routing of network traffic found in a conventional network edge routing device (also called an intelligent edge device) and the other scenario exhibits routing of network traffic but with an enhancement that incorporates fuzzy logic to the priority control mechanism of the intelligent edge device.

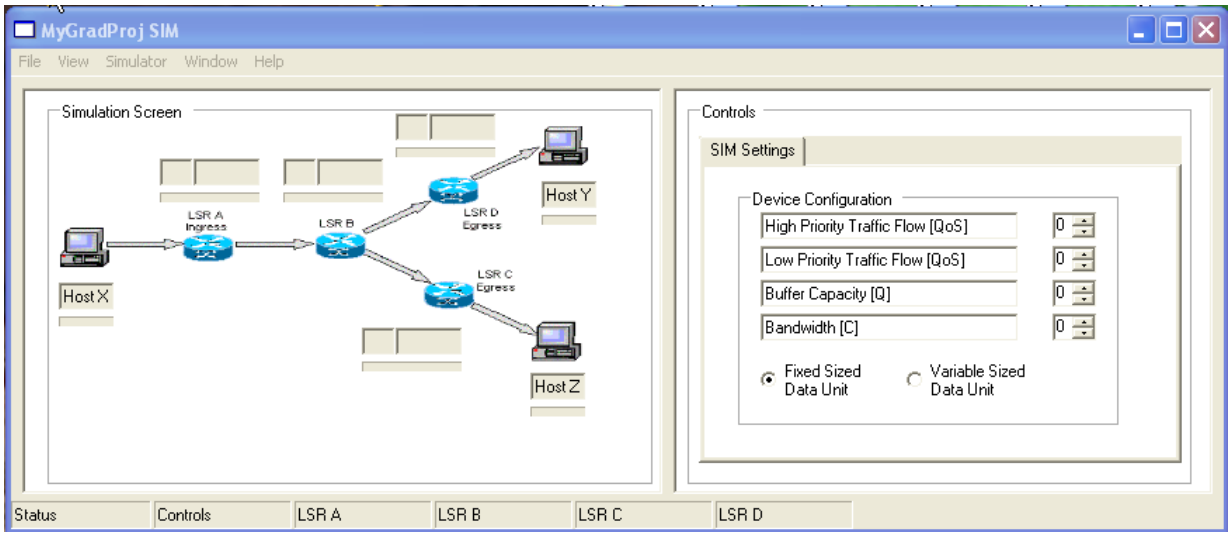


Figure 1 – Simulation screen with no fuzzy logic controls

The contrast between these scenarios' results demonstrates that this simulation model (by incorporating fuzzy logic in a conventional network edge routing device) results in a much higher degree of performance than with one found where no fuzzy logic was implemented. Even though the simulation model just emulates two very simplified and conceptual scenarios, the

software application does enable the user to experiment by making changes in the control parameters and seeing how these changes affect the overall simulation model (see Figure 2).

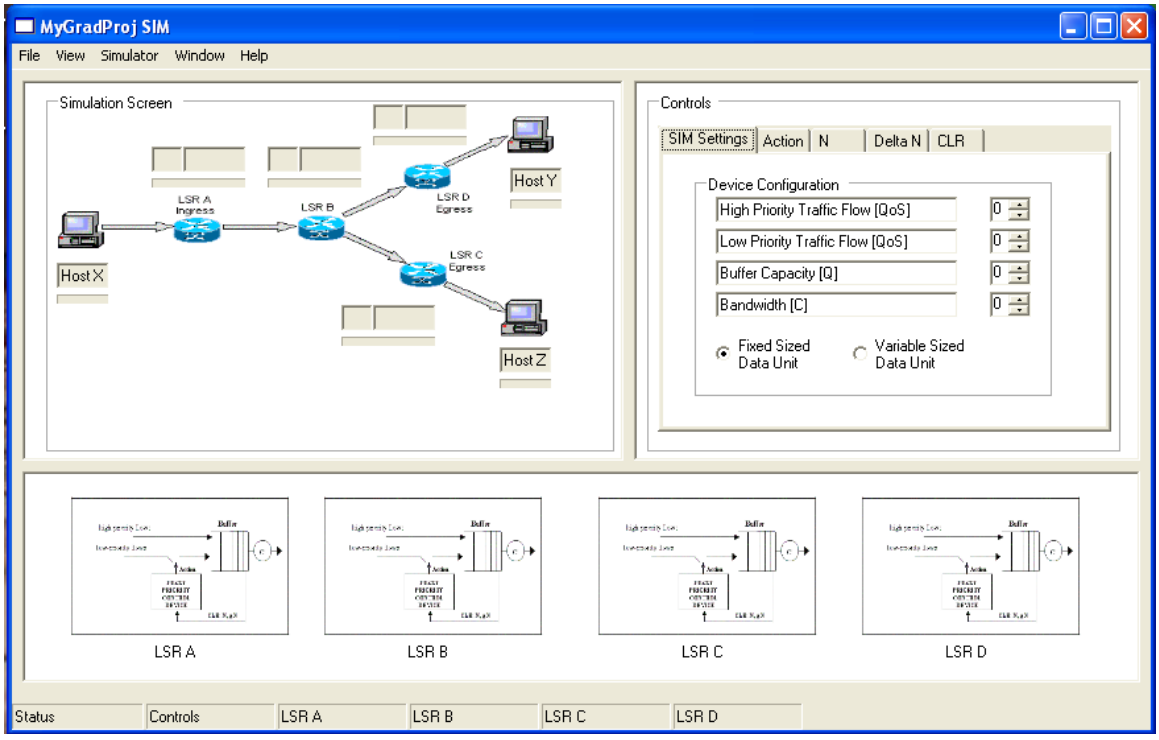


Figure 2 – Simulation screen with fuzzy logic controls

The simulation model demonstrates that an intelligent edge device that incorporates fuzzy logic leads to each device's throughput to increase the number of high priority packets within the network. Also, by demonstrating that the enhanced support found by combining fuzzy logic, the principle of Quality of Service (QoS), and an enhanced version of Multi-protocol Label Switching (MPLS), the simulation model can be used for experimenting with setting different combinations and analyzing the results. Additional User Interface (UI) specification details can be found in Appendix C.

SIGNIFICANCE OF THE PROJECT

Applying fuzzy logic to edge network devices combined with other enhancements is an exciting field with many possible directions. I have a good grasp of the state of the art from my work experience combined with research in past courses and with this project, I hope to generate and encourage more interest in this field of study.

An interactive simulator software application allows a user to explore and ask whether or not the application of fuzzy logic has any effect on network performance.

LIMITATIONS OF THE PROJECT

There are a number of limitations of the project that must be identified—finite routing paths, primitive router and host behavior, and a simulation model that does not support concurrency. First, by minimizing the number of routing options to having only three possible route directions, it simplifies the comparison made within the simulation between the non fuzzy logic and fuzzy logic approaches. Second, the representation of both the routers and hosts is based on very primitive programming constructs (e.g. data structures representing single linked list and queue). Finally, the simulation model does not support real world concurrency in order to simplify the model.

However, all of these limitations can be overcome given a significant amount of time and effort that can expand on this very simplified version of the SIM software to make it a more realistic representation found in the real world.

DEFINITIONS OF TERMS

EDGE DEVICE – A physical device that can pass packets between a legacy type of network and an ATM network, using Data Link Layer and Network Layer Information (an example of an edge device is an edge router).

MULTI-PROTOCOL LABEL SWITCHING – a short fixed-length label that represents an IP packet's header with subsequent routing decisions made based on the MPLS label and not the original IP address.

QUALITY OF SERVICE – refers to a broad collection of networking technologies and technique where the goal is to provide guarantees on the ability of the network to deliver predictable results.

FUZZY LOGIC – a type of logic that recognizes more than simple true and false values with propositions that can be represented with degrees of truthfulness and falsehood.

INGRESS – the act of entering.

EGRESS – the act of leaving.

CENTRALIZED NETWORK – a network topology where a central set of servers control both services and information.

DISTRIBUTED NETWORK – a network topology where a distributed set of servers control both services and information.

ROUTER – a device that forwards data packets along networks and it must be connected to at least two networks.

NETWORK SWITCH – a device that joining multiple computers together at a low-level network protocol level as well as operates at either layer two or layer three.

DE-FUZZIFICATION – a method to return a single crisp value from a set of fuzzy values.

PACKET – a piece of a message transmitted over a packet-switching network where it contains a destination address in addition to the data.

ATM – a network technology based on transferring data in cells or packets of a fixed size.

CELL – used with ATM and it is also referred to as packets of a fixed size.

REVIEW OF RELEVANT LITERATURE

A network's edge is arguably one of its most important areas, because this is where it delivers services to network users—or, from the Internet service provider perspective, to subscribers. Current TCP/IP congestion control algorithms cannot efficiently support new and emerging services needed by the Internet community [1]. In fact, one could argue that network congestion control still remains a critical and high priority issue especially given the growing size, demand, and speed (bandwidth) of the increasingly integrated services network [2]. These devices at the network's edge classify, prioritize, and mark packets for the rest of the network to understand, and ultimately allow them into the network. A proposal to effectively address this issue is in implementing a fuzzy logic controlled queue that is not only novel, more effective, robust, and scalable, but it is also a more flexible approach. So given modern networks' complex internetworking, it is not surprising that identifying, developing and incorporating fuzzy logic into edge devices in order to minimize routing congestion are tasks that head the agendas of most of today's network equipment manufacturers, carriers, service providers, and network administrators. Many network equipment manufacturers are now providing, as part of their product line, two different classes of network routers—core router and edge router (see Table 1) [3].

Table 1 – World market breakup for the public telecommunications infrastructure in 2001

Segment	Market size
Core routers	$1.7B
Edge routers	$2.4B
SONET/SDH/WDM	$28.0B
Telecom MSS	$4.5B

This manuscript begins with a brief background on fuzzy logic and then proceeds to define an edge device in the context of a network topology. Next, it is necessary to show how the quality of service principle supports the edge device's capability to provide differentiated services on the

network's edge. Finally, an enhanced version of MPLS routing combined with fuzzy logic purports the notion of blurring the network's edge with the implementation of fuzzy enabled edge devices.

FUZZY LOGIC BACKGROUND

Fuzzy logic is a technology for developing intelligent control and information systems. There are three areas of fuzzy logic from the viewpoint of machine intelligence, control systems and information technology. These three areas include (1) a way to represent and structure human knowledge that is imprecise by its very nature, (2) a very useful design technique that can be applied to nonlinear control systems where the basic building block consist of "if-then" rules (also known as fuzzy rule-based models) and used to approximate traditional functional mapping, and (3) an increasingly important aspect of storing and retrieving imprecise information from databases and information sources within advanced information systems [4]. Fuzzy logic can be viewed as a superset of conventional (Boolean) logic that has been extended to multi-valued logics. Fuzzy logic is an approach to computing based on "degrees of truth" rather than the usual "true or false". The idea of fuzzy logic was first proposed by Dr. Lotfi Zadeh of the University of California in Berkeley (in the 1960s) [18]. At that time, Dr. Zadeh was working on the problem of computers' understanding of natural language [19]. Natural language (like most other activities in life) is not easily translated into absolute terms such as "true or false". In fact, fuzzy logic seems closer to the way our brains work; that is, we aggregate data and form sets of partial truths that we in turn aggregate further into higher truths once certain thresholds are exceeded [5].

THE EDGE DEVICE

Cisco [1] defines an edge device as

> "...a physical device that can pass packets between a legacy type of network such as an Ethernet network and an asynchronous transfer mode (ATM) network, using data link layer (OSI Layer 2) and network layer (OSI Layer 3) information. An edge device does not have responsibility for gathering network routing information, but simply uses the routing

information it finds in the network layer using the route distribution protocol."

For example, an edge router (sometimes called a boundary router) routes data between one or more local area networks (LANs) and an ATM backbone network—whether it is a campus network or wide-area network (WAN). In this case, the edge device has the difficult task of aggregating, forwarding, and routing traffic.

We can contrast the edge router (which forwards packets to computer hosts within a network) with a core router,(which forwards packets between networks). A core device or core router is part of the backbone, and it serves as the single pipe through which all traffic from peripheral networks must pass on its way to other peripheral networks. Unlike core routers, which primarily switch packets, edge devices not only handle basic IP (Internet Protocol) packet processing, but also consolidate multiple wire-speed traffic streams into flows with appropriate QoS parameters, perform traffic conditioning, and provide a multiplicity of services (see Table 2 for a comparison of some of the characteristics and functions of core and edge devices) [6].

Table 2 – Core and edge device characteristics

Device type	Throughput (Gbps)	Functions
Core	80 to 320	Packet forwarding Route processing
Edge	6 to 24	Packet forwarding Route processing Traffic shaping QoS scheduling Service features

Another example of an edge device is an edge switch (refer to Figure 3, an example of an edge switch configuration within a network) [6]. These operate in various edge networks that interconnect through a high-speed, high-capacity core network which, in turn, connects to the outside world.

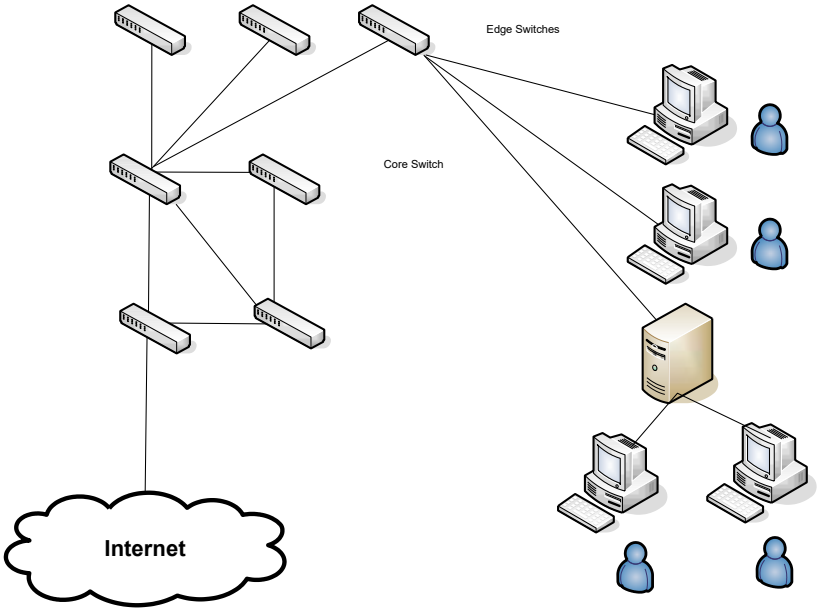


Figure 3 – Edge Switch Configuration within a Network

In addition to edge routers and switches, other types of edge devices include aggregation routers, service delivery, next-generation voice, and multi-service provisioning platforms. Many edge devices, designed to bridge the gap between consumer demand and core capacity, play more than one of these roles. A single device might incorporate the functions of an aggregation router, IP service switch, and carrier-class router; serving all these functions would require high degrees of reliability, flexibility, scalability, and performance.

For example, to terminate and aggregate diverse traffic from the network, edge devices must support various interfaces and speeds, and they must also manage hundreds of thousands of individual concurrent sessions, each with a unique profile—T1/E1 channel groups, frame relay sessions, ATM private virtual circuits (PVC), Ethernet virtual local area networks (VLAN), and so on.

THE QUALITY OF SERVICE PRINCIPLE

QoS is a key network principle for the transmission and distribution of digitized audio/video across next-generation high-speed networks. It has two objectives: finding routes that satisfy the QoS constraints and making efficient use of network resources. In this fashion, QoS mechanisms provide the necessary level of service (bandwidth and delay) to an application in order to maintain an expected quality level. To a mission-critical application, QoS means guaranteed bandwidth with zero frame loss. Fine-grain control provided by QoS places a significant burden on the network infrastructure. Each device must keep an entry in its forwarding table for each flow. In a large corporate network, devices can become overwhelmed with the millions of flows, especially at the boundaries. For example, MPLS is a protocol that delivers a level of service by application flow that could combine fuzzy logic with QoS to allow multiple constraints to be considered in a simple and intuitive way [7].

DIFFERENTIATED SERVICES ON NETWORKS

Increasing demand for inter-network access, technology advancements at both the desktop level and at the network level, and the increasing use of multi-media as well as other interactive technologies, are burdening the core network devices and LAN/WAN boundary routers in busy corporate networks. Service quality will continue to suffer without QoS mechanisms or traffic based classification schemes, such as differentiated services.

In response to demand for a robust, common system for service classification, an IETF Working Group drafted a framework and definitions for a differentiated services mechanism [8]. Differentiated services are not based on priority, application, or flow, but on the possible forwarding behaviors of packets, called Per Hop Behaviors (PHBs).

Unfortunately, in today's corporate network, QoS is a costly and difficult implementation. Differentiated services can alleviate bottlenecks by more efficient management of current corporate network resources. A differentiated service is a policy-based management tool for networks. By mapping multiple flows to a few service levels, differentiated services reduce the burden on network devices and easily scale with network growth.

Since a differentiated service is rule based, it is a unique mechanism for policy-based network management (making it a natural candidate for fuzzy logic). Instead of applying faster, more expensive, advanced technology, networks can be managed by appropriate network policies, applying current network technology and resources while considering in-house traffic and upstream and downstream networks, whether they are the corporate LAN backbone or external WANs.

For example, a differentiated service is analogous to travel services. A person can travel by bus, train, or airplane; first class, business class, coach, or standby. Each class of service can be characterized by how fast you reach your destination, how many stops you make along the way, and what kind of service amenities received en route, if any. Some services may have limitations, such as when you can travel, and others, such as standby, include risk of not reaching your destination in the time frame expected. In all cases, you pay more for higher quality services. The differentiated services framework offers the same kind of classification system. Based on network policies, different kinds of traffic can be marked for different kinds of forwarding whereby resources can then be allocated according to the marking and the policies.

NETWORK TOPOLOGIES

In considering how best to deploy edge devices, we should examine two network architecture types—centralized and distributed. From these two architecture types, network administrators derive three major topological approaches—centralized, a mixture of centralized and distributed, and distributed.

CENTRALIZED NETWORK

The centralized network topology assumes a collapsed backbone—a backbone network in which an intelligent edge device forwards a series of frames while the central device analyzes each frame and makes decisions (refer to Figure 4, an example of a centralized network [1]). From the network management perspective, this approach is relatively easy to implement. However, it means dealing with frames at a very high line rate—10 to 100 times faster than with a

distributed approach. The quantity of policies and flows that the backbone must support is dramatically higher, making performance unpredictable. Lack of redundancy is another shortcoming of this approach; the central device becomes a single point of failure. A centralized network management device distributes the set of rules by which these devices operate. The central devices are pipe only, which means that they have high-performance switching and queuing capabilities but do limited analysis. For example, in a Multi-protocol Label Switching approach, edge devices do all the flow treatment, and add or extract labels. The core devices simply perform label swapping and forward frames.

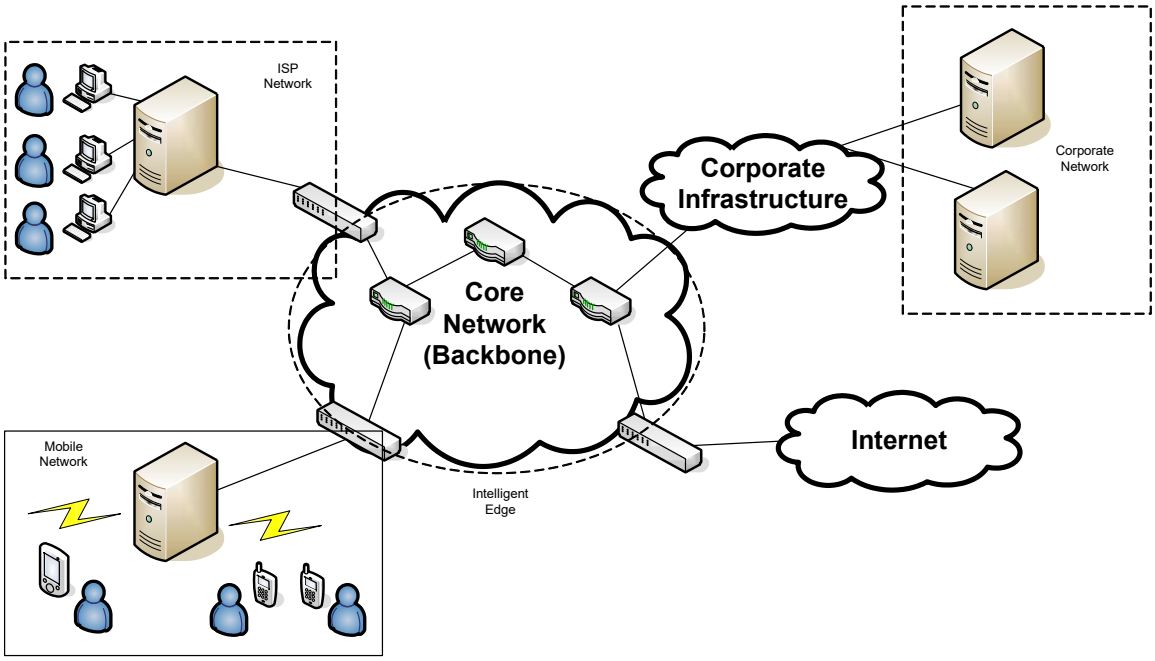


Figure 4 – Example of a Centralized Network

DISTRIBUTED NETWORK

A distributed network (refer to Figure 5, an example of a distributed network) that uses intelligent devices at the network edge is the preferable approach [6]. These devices perform frame analysis, classification, remarking, tagging, traffic shaping, and routing. They also perform analysis and decision making, though at relatively slower speeds than those of centralized

network devices.

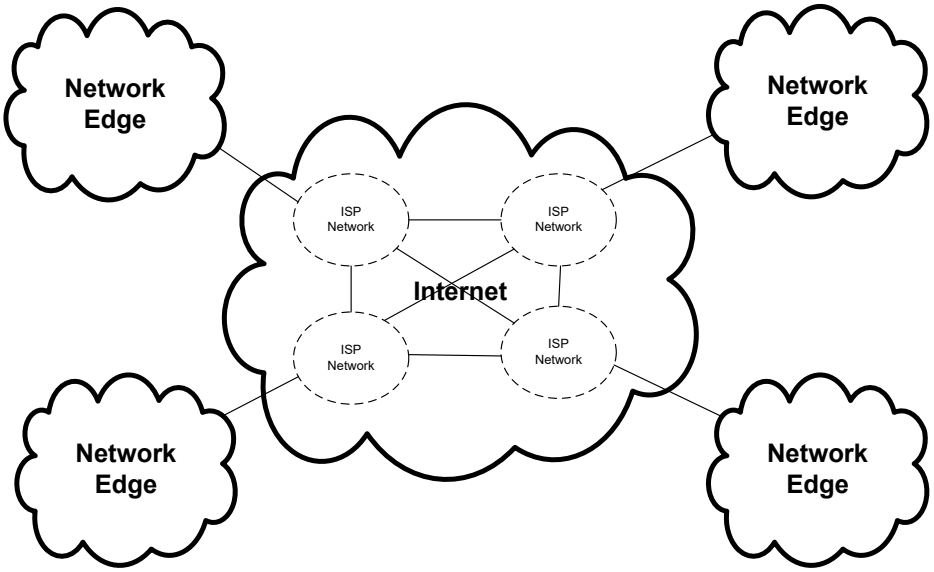


Figure 5 – Distributed network example

Although the centralized approach is easier to implement, the distributed network is more scalable, less sensitive to network failures, and has no single point of failure. Distributed networks are more cost-effective and yield more robust networks, with significantly better overall performance and efficiency [6].

COMBINED CENTRALIZED AND DISTRIBUTED NETWORK

The combined centralized and distributed topology assumes that the edge device is not simple and can do some classification and policy treatment between the backbone node and the edge device. However, the combined approach suffers from multiple areas of weakness [9]:

- it adds significant complexity and cost to the network without any appreciable gains—there are more nodes to install, manage, and support;
- the topology requires a new type of centralized device in the network architecture—a device has the same limitations as the central device in the centralized-network approach with no increase in benefits;
- sharing QoS responsibilities between edge and middle devices that do not conform to the well defined MPLS approach (Multi-protocol Label Switching, described in the next section)

leads to configuration problems and inefficiency—the network effectiveness deteriorates because some fields of the packets must be inspected twice (devices in the network must implement both QoS and traffic standards burdening the network with additional traffic management overhead); and

- the middle device handles intensive queue management and flow-related activities—as a result, it adds latency and sometimes unpredictable jitter that adversely affects multimedia applications.

MPLS ROUTING

Currently there are two major applications of MPLS in IP networks: traffic engineering (TE) and virtual private networks (VPNs) [10]. Traffic engineering is best viewed as the combination of functions that allows a network provider to control and monitor bandwidth and paths for traffic flows. It encompasses capacity planning, routing control, traffic management and path management. MPLS enables network-based IP-VPNs, a service offered by the network service provider that appears to the customer as a private network. VPNs benefit from MPLS because of the notion of tunnels.

The Internet Engineering Task Force [8] established the MPLS standards to achieve several capabilities, a few of which are

- Fast forwarding speed
- Traffic engineering, including constraint-based routing, explicit routing, and the abilities to compute a path at the source, reserve network resources, and modify link attributes
- Support for voice and video on IP, including the ability to handle delay variation plus QoS constraints
- Support for virtual private networks, including the use of a controllable tunneling mechanism, equivalent to a frame relay or ATM virtual circuit (VC)

As MPLS uses only the label to forward packets, it is protocol-independent, hence the term 'Multi-Protocol' in MPLS. MPLS uses IP addresses (either IPv4 or IPv6) to identify end points and intermediate switches and routers. The IP-compatibility and easy integration with traditional IP

networks make MPLS a universally accepted de facto standard as the flow-management model of choice [11]. However, unlike traditional IP, an MPLS flow is connection-oriented and its packets are routed along pre-configured Label Switched Paths (LSPs). In an MPLS network, first an edge–label-switching router (edge LSR) or *ingress router* assigns each incoming packet a label where the packet's destination address determines which LSP to use. Second, the router forwards the packet along a label-switched path, where each LSR makes forwarding decisions based solely on the label contents. Third, at each hop, the LSR strips the existing label and applies a new one that tells the next hop how to forward the packet. Finally, as the labeled packet leaves the network, another edge LSR (*egress router*) removes the label (refer to Figure 6, an example of an MPLS network) [12]. In addition, as the priority algorithms have to act on the forwarding path of the LSR, they cannot be excessively complex given the requirement to execute as rapidly as possible. It is here within the LSR priority algorithm mechanism that fuzzy logic is a natural application candidate.

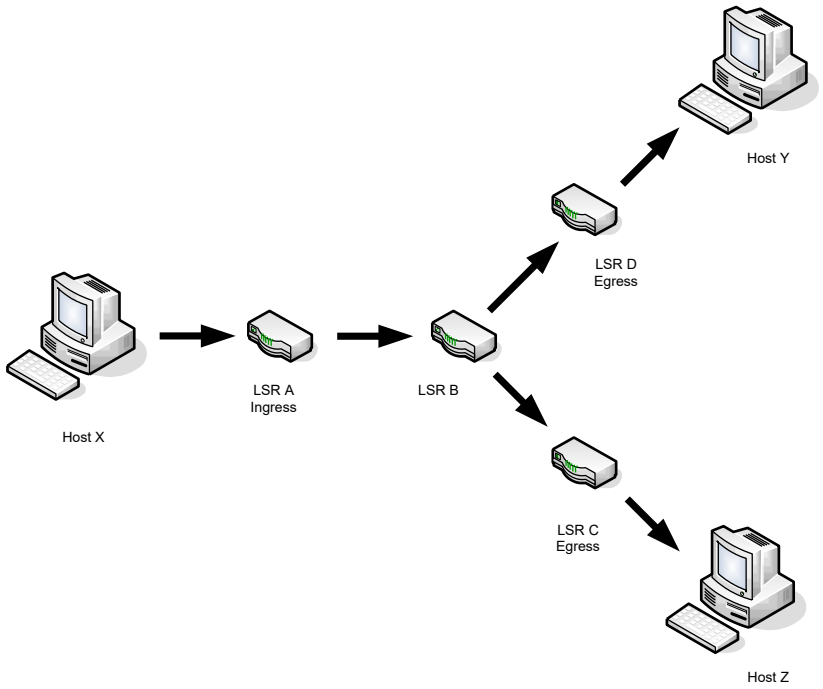


Figure 6 – An MPLS network example

For example, one can use MPLS to forward packets over the Internet backbone through the use of a VPN—through either border gateway protocol (BGP) or MPLS VPNs in either OSI layer 2 or layer 3. Used together, the MPLS signaling and VPN protocols establish a VPN tunnel, also known as label-switched paths, through which data travels. In addition, you can set up the MPLS label edge router to maintain multiple tunnels simultaneously.

ENHANCED MPLS ROUTING

A newly published RFC (available online) titled “Requirements for Support of Differentiated Services-Aware MPLS Traffic Engineering” (RFC3564) describes various application scenarios identified by Service Providers where existing MPLS traffic engineering (TE) mechanisms fall short and where differentiated services aware TE can address the needs. In fact, MPLS with guaranteed bandwidth services are extending MPLS traffic engineering—advertise available bandwidth for best-effort traffic and advertise available bandwidth for high priority traffic or differentiated services aware TE by utilizing QoS features to guarantee delivery of the high priority traffic (that includes classification with policing) [12].

For example, the corporate network can be divided into a core and edge topology configuration where the core network provides guaranteed bandwidth services (assuring value-added services with traffic engineering) and the edge of the network consists of existing MPLS TE classification with policing to ensure that there is no theft of the edge service provided to the customer. So, by defining a priority control scheme based on fuzzy logic (that respects the QoS of high priority traffic) combined with enhanced MPLS routing (that includes differentiated service awareness) applied to a traditional edge device results in a fuzzy enabled edge device.

THE FUZZY ENABLED EDGE DEVICE

The objective of a fuzzy enabled edge device (attached to the Internet) is to increase network capacity and offer a high quality of differentiated services for traffic with real-time and non-real-time requirements. An assumption made for Ma et al’s proposed solution [13] is that some switch resources (e.g. buffer size, Q, and bandwidth portion, C) are allocated to high-priority traffic to

guarantee a given QoS. To help clarify this point, a reference model is given in Figure 7 where low-priority cells are allowed to enter the allocated buffer to improve the exploitation of reserved resources [13]. This solution incorporates a fuzzy control algorithm to select packets in a fair and efficient manner and is ultimately implemented in a hardware based solution (for instance, implementation would come in the form of exploiting the advantages of VLSI technology) where the processor could achieve optimal performance in both processing capacity and latency [14].

In order to meet the demand for QoS threshold (for a particular real-time or non-real-time requirement), the differentiated service relies on a priority control function to maximize network utilization without degrading the QoS for high-priority traffic. The existence of low-priority packets could be due [14]: (1) to the tagging action of traffic enforcement mechanisms; or (2) to the traffic of an agency for which a certain QoS is guaranteed. The agency could exploit the resources allocated to real-time traffic (high-priority flow) to transmit best-effort traffic (low-priority).

The fuzzy logic based priority control function proposed by V. Catania et al. [14] consists of three input variables, one output variable, membership functions, 80 fuzzy rules, and an inference method. The three input parameters, the Cell Loss Ratio experienced in the buffer by High-priority traffic (CLR), the number of cells in the buffer (N) and the rate at which the length of the queue in the buffer varies or delta N (DN), are considered to be linguistic variables. There is one output variable called Action (a discrete value) and it is composed of six fuzzy sets whose names are: Strong Discard (SD), Discard (D), Marginal Discard (MD), Marginal Admit (MA) Admit (A) and Strong Admit (SA). There are a total of four membership functions where there is one membership function per parameter—CLR, N, DN, and Action (refer to Figures 8, 9, 10, and 11 for examples of four the membership functions) [14].

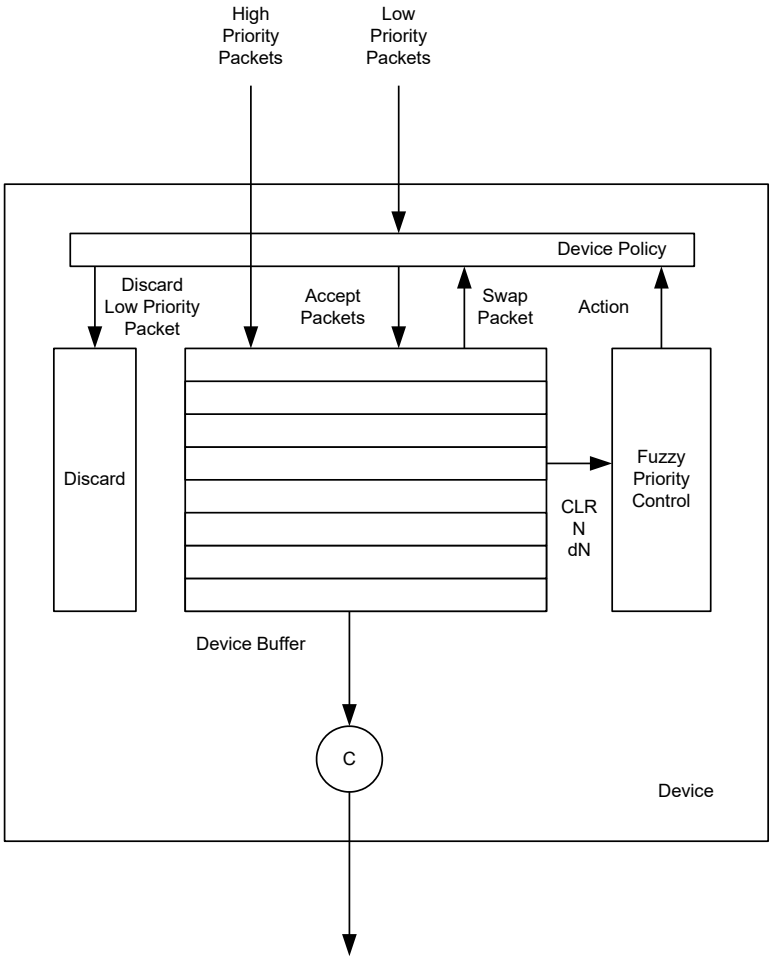


Figure 7 – An example of a reference model that incorporates a fuzzy priority control device where linguistic variables are the inputs and Action is the output variable

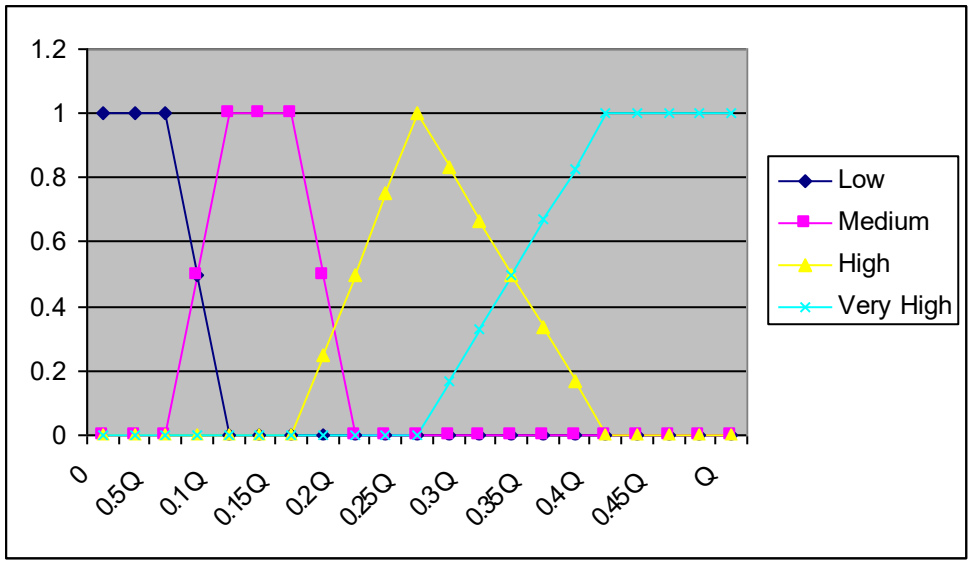


Figure 8 – An example of a membership function for the N input variable

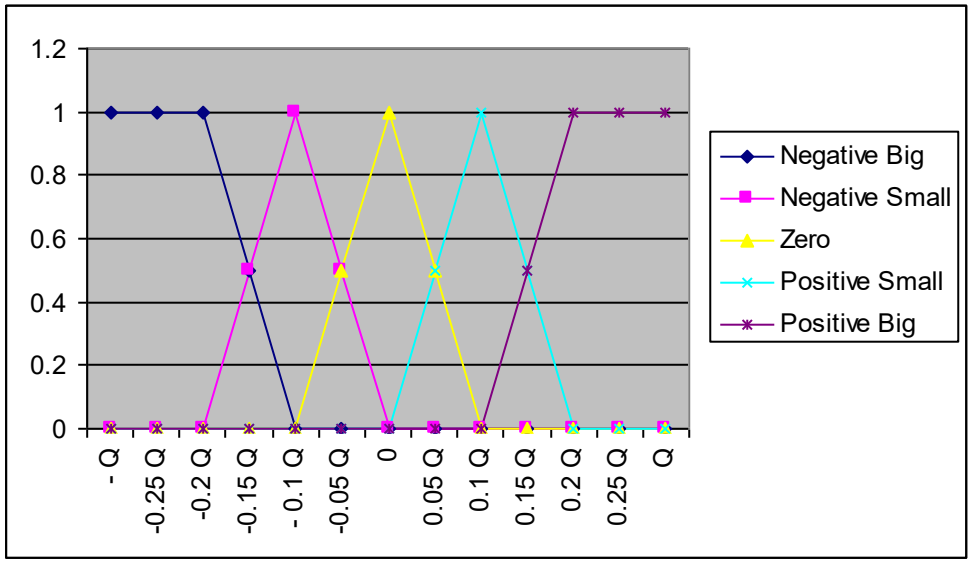


Figure 9 – An example of a membership function for the DN input variable

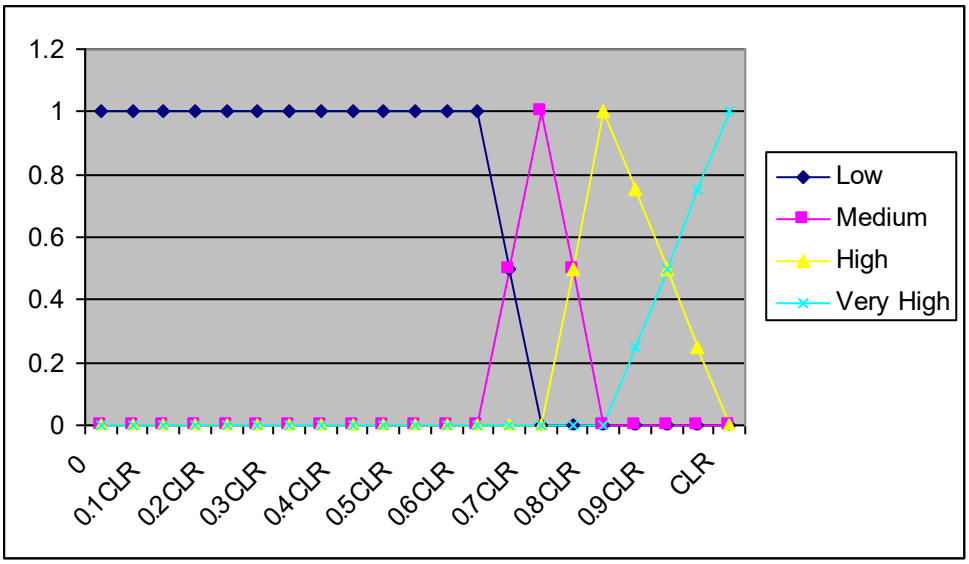


Figure 10 – An example of a membership function for the CLR input variable

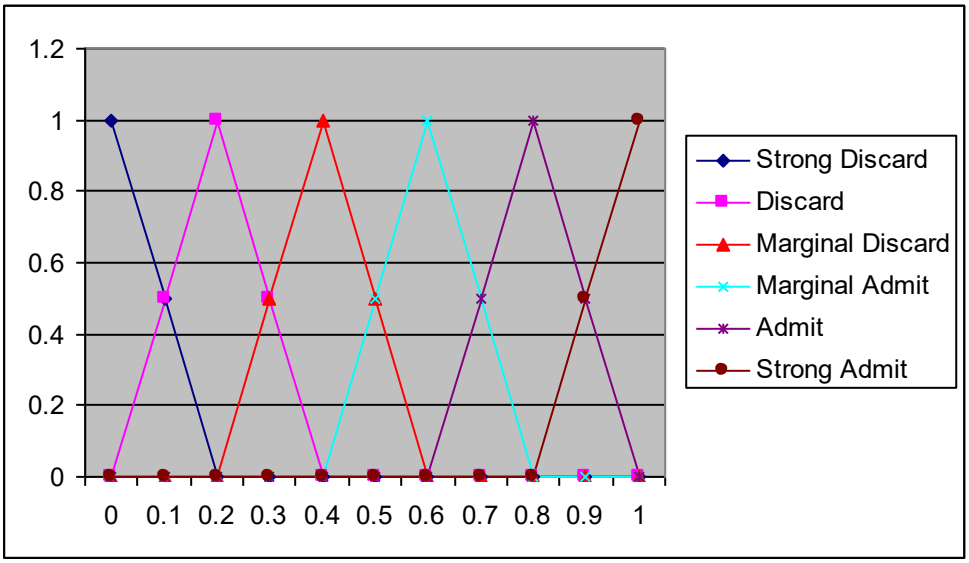


Figure 11 – An example of a membership function for the Action output variable

A requirement when defining fuzzy rules is to make them as simple as possible. In this case, even though there are over 400 combinations possible, the priority control function requires only a total of 80 fuzzy rules that are highly representative of the decision-making process (refer to Table 3 found in Appendix A, for a list of fuzzy rules that represent the fuzzy priority control

scheme) [14]. The fuzzy rules are based on intuitive considerations and then tuned by simulation test using the Genesis simulator with a Genetic Algorithm approach [14].

The inference method is Max-Min and for defuzzification the Center of Gravity (COG) approach is recommended [14]. Inference supplies a membership function as a result. A representative value cannot directly process this fuzzy information therefore the result of the inference process has to be converted into crisp numerical values. In this context, the crisp number to be determined should provide a good representation of the information contained in the membership function. The Center of Gravity approach addresses the problem of comparison of fuzzy numbers and it is to associate with a fuzzy number F some representative value, Val (F), and to compare the fuzzy subsets using these single representative values [15]. In addition, the equations can be found in Appendix D.

BLURRING THE NETWORK'S EDGE

The network edge is no longer a simple demarcation point within a particular network; rather, it must contain some of the intelligence to meet the requirements of new technologies and of the business and regulatory environments. If customers are to receive reliable, affordable service—and if providers are to do business in a way that generates profit—network administrators need new tools to efficiently manage devices at the network edge [20].

Today, a new kind of IP network is evolving to meet the converging needs and technologies of several market segments—customers, ISPs, corporations, traditional telephony carriers, and other new types of customer-facing service providers. And the place where these market segments come together is the network edge. Understanding the important aspects of network topologies, fuzzy enabled edge devices, and enhanced MPLS routing can help guide the network's evolution to participate effectively and profitably in this convergence and even blur it to the point of a ubiquitous network. For example, enhanced MPLS routing can provide edge-assisted QoS to networks where in the past only the edge routers can participate in QoS tagging. This protocol combines differentiated services and call-admission control function that sets up a gatekeeper for call admission and establishes priority classes for IP traffic and in a sense blurs the network's edge as a conventional demarcation point [16].

CURRENT FUZZY ENABLED EDGE DEVICE USES

The evolving edge device is an extremely versatile especially with the application of fuzzy logic. It must support several transport technologies and interface speeds to provide aggregation and connectivity among different network topologies and elements. Such fuzzy enabled edge devices also support an increasing number of complex protocols and packet encapsulations to manage diverse network topologies and deliver new services and applications [17]. With the growing popularity of broadband services and third-generation, mobile wireless devices, tomorrow's edge devices utilizing differentiated services traffic engineering and enhanced MPLS will have to scale to even larger table loads.

These forces are driving the modern network architecture to a more hierarchical based structure with a fast, converged core combined with an intelligent edge [17]. As network devices become more specialized (to support a more hierarchical based structure) and begin to incorporate intelligent systems techniques such as fuzzy logic into existing network devices (e.g. routers, switches, and aggregation multiplexers) a new breed of network device will emerge. Tomorrow's fuzzy logic enabled edge device will function much more effectively than its predecessor (e.g. Ethernet, frame relay, and ATM edge switches; IP service platforms; and carrier-class routers). We can anticipate that by exchanging information between the network core and broadband access networks, these sophisticated edge devices will not only simultaneously support widely diverse protocols, interfaces, and line rates; handle heavy control- and data-plane loads; but will also provide security, QoS, and many other IP services—all at wire speed [17].

METHODOLOGY

The work presented here aims to provide a contrast between two scenario results. A simulation model is used to demonstrate incorporating fuzzy logic principles in a conventional network edge routing device, the result is a marginal increase in performance than one with no fuzzy logic implementation. Even though the simulation model just emulates two very simplified

and conceptual scenarios, the software application enables the user to experiment by making changes in the control parameters and displays how these changes affect the overall simulation model (refer to Figure 12).

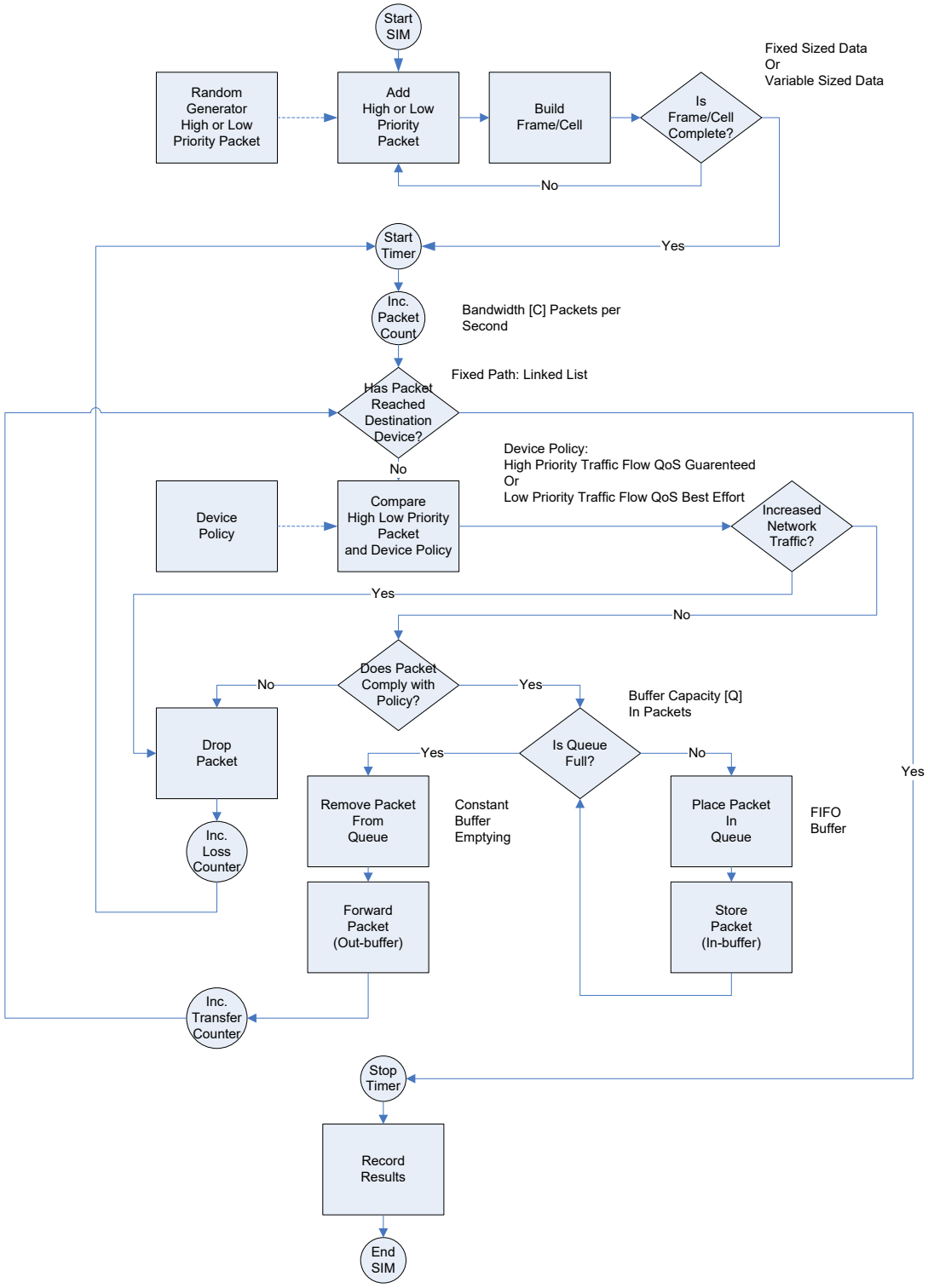


Figure 12 – Simulator Process Flow without Fuzzy Logic Applied

The fundamental design is to optimize the amount of high priority traffic that is routed through each device along a path and increase the overall throughput of each of the fuzzy logic enabled devices. When the inbound queue of the device is not full every inbound packet, regardless of its priority is accepted but when the queue is full and a low-priority packet arrives, it will be discarded. Conversely, when a high-priority packet arrives the low-priority packet is removed from the inbound queue (of course if a low-priority packet is in the inbound queue). If there are no low-priority packets waiting, the arriving packet is discarded (refer to Figures 13 and 14).

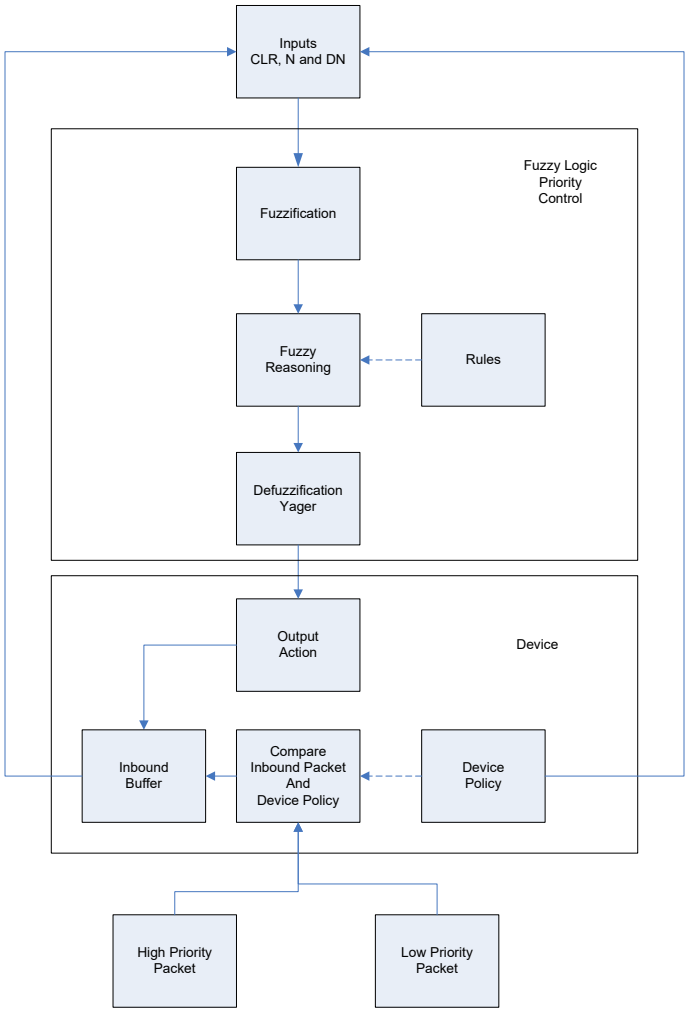


Figure 13 – Fuzzy Logic Priority Flow Process

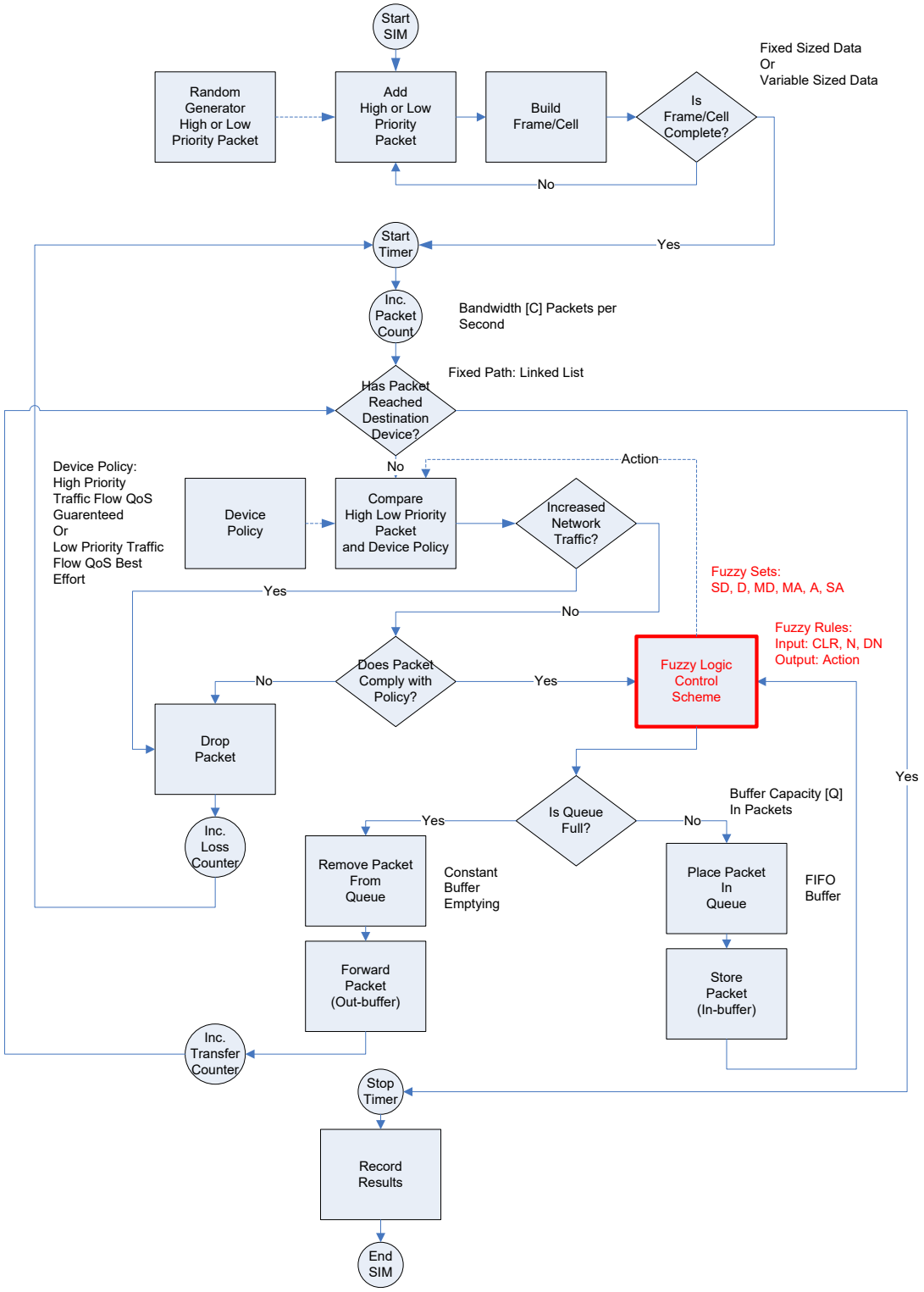


Figure 14 – Simulator Process Flow with Fuzzy Logic Applied

The software application was written using the Microsoft Visual C# .NET development platform. It runs on Microsoft Windows 2000 and XP desktop operating system as a client software application. In order to not recreate a brand new fuzzy model with the SIM, an available open source fuzzy logic class library and API was used. The Free Fuzzy Logic Library (FFLL) (**http://ffll.sourceforge.net**) is an open source fuzzy logic class library and API that adheres to the IEC 61131-7 standard [20]. The FFLL was compiled using a different version of complier than the project adding to the completing of this project. The FFLL was compiled on the Microsoft Visual C++ development platform and then linked with this project.

The FFLL class library and API provided all the necessary components for a working fuzzy logic model. The fuzzy variables contain sets where each set has a membership function associated with it. The membership function determines the set's shape in this case where its use is to "fuzzify" the x values of the variable with the associated Degree of Membership (DOM). One of the most common membership function used in fuzzy systems is a triangle however, FFLL provides support for Triangles, Trapezoids, S-Curves, and Singletons. Each set's membership function contains a lookup table used to speed the "fuzzification" process (however even though lookup tables are fast the tradeoff is increased use of memory). The variable's x-axis values must be mapped into to the values array. FFLL stores rules in a one-dimensional array. For example, in a 3x3x3 system (three input variables, each with three sets) there would be 27 total rules. To eliminate extra calculations, FFLL does not calculate the defuzzified output value until it is requested. There are two types of defuzzification approaches available in FFL. The Center of Gravity (COG) defuzzification approach makes heavy use of lookup tables while the Mean of Maximum (MOM) defuzzification approach simply stores a single value per output set. The defuzzification approach used for this project was COG.

The API is straight forward in that it provides seven functions—create a new fuzzy logic model, close the model, load a fuzzy model from a file (based on the IEC 61131-7 Fuzzy Control Language or FCL format, create a child helper process for the model, set input variable in a child, get defuzzified output value for a child and get message text when an API function returns an error.

The Fuzzy Control Language or FCL syntax (based on the standard for Fuzzy Control Programming published by the International Electrotechnical Commission or IEC) can be found within the IEC specification document 61131-7 [15]. FFLL maintains only the minimum compliance necessary with IEC specification document 61131-7 (refer to a table of minimum FCL compliance and a listing of production rules in Appendix E).

RESULTS

By combining MPLS protocol, fuzzy logic and an edge network device, I purport that network traffic can be prioritized and routed more effectively whereby demonstrating a marginal increase in the total throughput at the edge of the network while meeting a QoS goal for high traffic demand.

Expected result was that by using fuzzy logic the overall throughput would increase with respect to low-priority traffic when high priority traffic is admitted and low-priority traffic is not admitted. In addition, if overall throughput increased then this implies that the network efficiency also increases as long as it is not at the expense of noncompliance with the device policy. The throughput was measured in terms of high priority traffic as a percentage of total traffic (high priority traffic and low priority traffic) transferred along a path of connected devices over a period of time.

Actual result averaged over five trials with no congestion applied and when no fuzzy logic was applied can be found in Table 4.

No Fuzzy Logic Applied with QoS

Trials	Total Throughput [Packets]	Total Throughput [Percent]	Low Priority Load [Packets]	High Priority Load [Packets]	Elapsed Time [Seconds]	Congestion [Percent]
1	50	56.00%	22	28	0.812	0%
2	53	52.83%	25	28	0.812	0%
3	56	50.00%	28	28	0.812	0%
4	56	50.00%	28	28	0.796	0%
5	56	50.00%	28	28	0.796	0%
Average	54	52%	26	28	0.8056	0%

Table 4 – Results when no fuzzy logic was applied to the system

Actual result averaged over five trials with no congestion applied and when fuzzy logic was applied can be found in Table 5.

Fuzzy Logic Applied with QoS

Trials	Total Throughput [Packets]	Total Throughput [Percent]	Low Priority Load [Packets]	High Priority Load [Packets]	Elapsed Time [Seconds]	Congestion [Percent]
1	56	50.00%	28	28	0.328	0%
2	56	50.00%	28	28	0.39	0%
3	56	50.00%	28	28	0.515	0%
4	56	50.00%	28	28	0.593	0%
5	56	50.00%	28	28	0.484	0%
Average	56	50%	28	28	0.462	0%

Table 5 – Results when fuzzy logic was applied to the system

It can be concluded from the limited results demonstrated that total throughput marginally increased when fuzzy logic was applied to the system. However, by increasing the number of trials the larger set of results would reinforce the conclusion.

A couple additional scenarios were considered—increasing high priority load QoS policy threshold and increasing congestion.

In the scenario where by increasing the high priority load QoS policy threshold, the number of high priority packets was greater than the number low priority packets but the total throughput remained steady.

Actual result averaged over five trials with high priority QoS set at 80% and no fuzzy logic was applied can be found in Table 6.

No Fuzzy Logic Applied with High Priority QoS set at 80%

Trials	Total Throughput [Packets]	Total Throughput [Percent]	Low Priority Load [Packets]	High Priority Load [Packets]	Elapsed Time [Seconds]	Congestion [Percent]
1	56	78.57%	11	44	0.859	0%
2	56	78.57%	11	44	0.796	0%
3	56	78.57%	11	44	0.796	0%
4	56	78.57%	11	44	0.828	0%
5	56	78.57%	11	44	0.828	0%
Average	56	79%	11	44	0.8214	0%

Table 6 – Results when no fuzzy logic and high priority QoS set at 80%

Likewise, in the same scenario but with fuzzy logic applied the number of high priority packets was greater than the number low priority packets but the total throughput remained steady. Actual result averaged over five trials with high priority QoS set at 80% and with fuzzy logic was applied can be found in Table 7.

Fuzzy Logic Applied with High Priority QoS set at 80%

Trials	Total Throughput [Packets]	Total Throughput [Percent]	Low Priority Load [Packets]	High Priority Load [Packets]	Elapsed Time [Seconds]	Congestion [Percent]
1	56	78.57%	11	44	0.843	0%
2	56	78.57%	11	44	0.765	0%
3	56	78.57%	11	44	0.718	0%
4	56	78.57%	11	44	0.625	0%
5	56	78.57%	11	44	0.625	0%
Average	56	79%	11	44	0.7152	0%

Table 7 – Results when fuzzy logic was applied and high priority QoS set at 80%

In the scenario where by increasing the overall congestion of the system, the results were found to be the same as with the prior scenarios and the total throughput was steady like in the prior scenarios. However, the high priority packets were given priority at the expense of the low priority packets with network congestions applied.

Actual result averaged over five trials with network congestion set at 20% and no fuzzy logic

was applied can be found in Table 8.

No Fuzzy Logic Applied with Network Congestion set at 20%

Trials	Total Throughput [Packets]	Total Throughput [Percent]	Low Priority Load [Packets]	High Priority Load [Packets]	Elapsed Time [Seconds]	Congestion [Percent]
1	35	80.00%	0	28	0.828	20%
2	47	59.57%	0	28	0.828	20%
3	29	96.55%	0	28	0.812	20%
4	32	87.50%	0	28	0.812	20%
5	38	73.68%	0	28	0.812	20%
Average	36	79%	0	28	0.8184	20%

Table 8 – Results when no fuzzy logic was applied and network congestion set at 20%

Fuzzy Logic Applied with Network Congestion set at 20%

Trials	Total Throughput [Packets]	Total Throughput [Percent]	Low Priority Load [Packets]	High Priority Load [Packets]	Elapsed Time [Seconds]	Congestion [Percent]
1	56	80.36%	0	45	0.843	20%
2	56	80.36%	0	45	0.937	20%
3	56	80.36%	0	45	0.609	20%
4	56	80.36%	0	45	0.656	20%
5	56	80.36%	0	45	0.781	20%
Average	56	80%	0	45	0.7652	20%

Table 9 – Results when fuzzy logic was applied and network congestion set at 20%

SUMMARY

This master's project covers a review of the network's edge because this is where the network delivers services to network users—or, from the Internet service provider perspective, to subscribers. Current TCP/IP congestion control algorithms cannot efficiently support new and emerging services needed by the Internet community [17]. In fact, one could argue that network congestion control still remains a critical and high priority issue especially given the growing size, demand, and speed (bandwidth) of the increasingly integrated services network [17]. These

devices at the network's edge classify, prioritize, and mark packets for the rest of the network to understand, and ultimately allow them into the network. In fact, the purported solution and its demonstration through a simulator software shows that an intelligent edge device incorporating fuzzy logic not only blurs the network's edge toward ubiquitous network topology but also enables enhanced support of complex protocols (packet encapsulations to manage diverse network topologies) in order to deliver new services. The enhanced support is developed by combining fuzzy logic, the principle of Quality of Service (QoS), and an enhanced version of Multi-protocol Label Switching (MPLS) to address many of the challenges facing today's networks. Given modern networks' complex internetworking, it should not be a surprise to anyone that identifying, developing and incorporating intelligent systems techniques such as fuzzy logic into the conventional edge device are at the top agenda item of most of today's network equipment manufacturers, carriers, service providers, and network administrators.

CONCLUSIONS

Applying fuzzy logic to edge network device combined with other enhancements is an exciting field with many possible directions. The simulator software developed for this project provides the user the ability to explore different combinations of fuzzy logic configurations applied to simplified network topology. The simulator software demonstrates that there is an advantage to using fuzzy logic over using a traditional device without fuzzy logic.

RECOMMENDATIONS

The limitations for the simulator software can be overcome given a significant amount of time and effort applied to its current model. The functionality can be expanded and complexities introduced within the simulator software to make it a more realistic representation of the real

world. Removing the cap on the number of routing options will facilitate more possibilities with real world network topologies for experimentation with throughput efficiencies. Second, the representation of both the routers and hosts can be made more like real devices by increasing the complexity of the programming constructs. Finally, the simulation model should support concurrency instead a single threaded programming model.

In order to balance the number of configurations between the network and fuzzy logic features within the simulator software a number of restrictions where implemented. However, a few of the possible configurations are restricted such as limited to three routing options, type of membership functions, number of membership functions, number of rules, type of t-norm/r-conorm, inference approach cannot be changed within the simulator software. The user can manipulate the configuration by selecting one of three networks paths, adjusting congestion, select all or none of the fuzzy logic membership functions and select all or none of the fuzzy logic rules.

REFERENCES

[1] Mauer, Lowell, and Perry, Greg. *Internetworking Technologies Handbook,* third edition, (Indianapolis: Cisco Press, 2000) p. 292.
[2] Chrysostomou C., Pitsillides A., Rossides L., Sekercioglu A., "Fuzzy logic controlled RED: congestion control in TCP/IP differentiated services networks." *Soft Computing* 8 (2003) Springer-Verlag 2003. pp. 79–92.
[3] Molinero-Fernandez, P., McKeown, N., Zhang, H., "Is IP going to take over the world (of communications)?" *ACM SIGCOMM Computer Communications Review* (2003) ACM Press 2003. p. 114.
[4] Langari, Reza, Yen, John. *Fuzzy Logic Intelligence, Control, and Information*. (New Jersey: Prentice Hall, 1999) p. 16.
[5] *The Berkeley Initiative in Soft Computing Home Page*. April 2004. Electrical Engineering and Computer Sciences Department University of California, Berkeley. 26 April. 2004. <**http://www.cs.berkeley.edu/~zadeh/pripub.html**>. 1st paragraph.
[6] Ginsburg, David, and Hattar, Marie, Implementing IP Services at the Network Edge, 1st Ed. (Boston: Addison-Wesley 2001), p. 10.
[7] Long, Keping, Zhang, Runtong "A fuzzy approach to the admission control in diffserv networks." *Proc. 2002 International Conference on Fuzzy Systems and Knowledge Discovery (FSKD'02),* volume 1, Singapore, December 2002. pp. 305-309.
[8] Behrouz A. Forouzan, TCP/IP Protocol Suite, ed. (New York: McGraw-Hill, 2000) p. 8.
[9] Livoli, Karen, "Deploying Service at the network Edge" IP Routing Group, Unisphere Networks, EE Times, May 6, 2002. <**http://www.eetimes.com/in_focus/embedded_systems/OEG20020503S0072**>. Path: Technical Paper. 1st to 13th paragraphs
[10] Alcatel Corporation. "The Role of MPLS Technology in Next Generation Networks", 1 Oct 2000. <**http://www.cid.alcatel.com/doctypes/techpaper/mpls/index.jhtml**>. Path: Technical Paper. pp. 3- 7.
[11] Data Connection Ltd. "What is MPLS?" 26 April 2004 <**http://www.dataconnection.com/mpls/whatis.htm**>. Path: Products. 4th to 8th paragraph
[12] Jamoussi, Bilel, and Rybczynski, Tony. "Powering Core Networks Through MPLS" Nortel Network, Internet Telephony, January 2001. pp. 1-3.
[13] Ma, Jian, Phillis A., Yannis and Zhang, Runtong. "A fuzzy approach to the balance of drops and delay priorities in differentiated services networks." *IEEE Transaction on Fuzzy Systems*, 2003. pp. 305-309
[14] Catania V., Ficili G., Palazzo S., Panno D., "A Fuzzy Buffer Management Scheme for ATM and IP Networks" Istituto di Inf. e Telecommun., Catania Univ., Italy. April 2001. pp. 1-9.
[15] IEC 1131 – Programmable controllers, Part 7 – Fuzzy Control Programming. Technical Committee No. 65: Industrial Process Measurement and Control Sub-Committee 65 B: Devices. 19 January 1997. pp. 11 – 35.
[16] Wirbel, Loring "Routing evolution a chief concern at Globecom show", EE Times, January 4, 2002. Path: Technical Paper. < **http://www.eetimes.com/**> 1st to 12th paragraphs
[17] Yocam, Eric. "Evolution on the Edge: Intelligent Devices." *IEEE Computer Society IT Professional*, volume 5, number 2, United States, March-April, 2003. pp. 24–27.
[18] Zadeh, Lotfi A., Linear System Theory-The State Space Approach, (co-authored with C. A. Desoer). New York: McGraw-Hill Book Co., 1963 pp. 338-353.
[19] Zadeh, Lotfi A., Probability measures of fuzzy events, Jour. Math. Analysis and Appl. April 23, 1968. pp. 421-427.
[20] Tan, Chiang K., "The Use of Fuzzy Metric in OSPF Network" August 2001. Department of Electronic and Electrical Engineering University College London, London. 31 August 2001. p. 4.

APPENDICES

APPENDIX A - Fuzzy Priority Control Scheme

Table 3: Fuzzy Priority Control Scheme

Rule #	Input: CLR	Input: N	Input: DN	Output: Action
1	Low	Low	Negative Big	Strong Admit
2	Low	Low	Negative Small	Strong Admit
3	Low	Low	Zero	Strong Admit
4	Low	Low	Positive Small	Strong Admit
5	Low	Low	Positive Big	Strong Admit
6	Low	Medium	Negative Big	Strong Admit
7	Low	Medium	Negative Small	Admit
8	Low	Medium	Zero	Marginal Admit
9	Low	Medium	Positive Small	Marginal Admit
10	Low	Medium	Positive Big	Marginal Admit
11	Low	High	Negative Big	Admit
12	Low	High	Negative Small	Marginal Admit
13	Low	High	Zero	Marginal Admit
14	Low	High	Positive Small	Marginal Discard
15	Low	High	Positive Big	Marginal Discard
16	Low	Very High	Negative Big	Marginal Discard
17	Low	Very High	Negative Small	Marginal Discard
18	Low	Very High	Zero	Discard
19	Low	Very High	Positive Small	Discard
20	Low	Very High	Positive Big	Discard
21	Medium	Low	Negative Big	Strong Admit
22	Medium	Low	Negative Small	Admit
23	Medium	Low	Zero	Admit
24	Medium	Low	Positive Small	Admit
25	Medium	Low	Positive Big	Marginal Admit
26	Medium	Medium	Negative Big	Admit
27	Medium	Medium	Negative Small	Admit
28	Medium	Medium	Zero	Marginal Admit
29	Medium	Medium	Positive Small	Marginal Admit
30	Medium	Medium	Positive Big	Marginal Admit
31	Medium	High	Negative Big	Marginal Admit
32	Medium	High	Negative	Marginal Admit

			Small	
				Marginal
33	Medium	High	Zero	Discard
				Marginal
34	Medium	High	Positive Small	Discard
				Marginal
35	Medium	High	Positive Big	Discard
		Very		Marginal
36	Medium	High	Negative Big	Discard
		Very	Negative	Marginal
37	Medium	High	Small	Discard
		Very		
38	Medium	High	Zero	Discard
		Very		
39	Medium	High	Positive Small	Discard
		Very		
40	Medium	High	Positive Big	Strong Discard
41	High	Low	Negative Big	Admit
			Negative	
42	High	Low	Small	Admit
43	High	Low	Zero	Marginal Admit
44	High	Low	Positive Small	Marginal Admit
45	High	Low	Positive Big	Marginal Admit
46	High	Medium	Negative Big	Marginal Admit
			Negative	
47	High	Medium	Small	Marginal Admit
48	High	Medium	Zero	Marginal Admit
				Marginal
49	High	Medium	Positive Small	Discard
50	High	Medium	Positive Big	Discard
51	High	High	Negative Big	Marginal Admit
			Negative	Marginal
52	High	High	Small	Discard
53	High	High	Zero	Discard
54	High	High	Positive Small	Discard
55	High	High	Positive Big	Discard
		Very		Marginal
56	High	High	Negative Big	Discard
		Very	Negative	
57	High	High	Small	Discard
		Very		
58	High	High	Zero	Discard
		Very		
59	High	High	Positive Small	Discard
		Very		
60	High	High	Positive Big	Strong Discard
61	Very High	Low	Negative Big	Marginal Admit
			Negative	
62	Very High	Low	Small	Marginal Admit
63	Very High	Low	Zero	Marginal Admit
64	Very High	Low	Positive Small	Marginal Admit
65	Very High	Low	Positive Big	Marginal Admit
66	Very High	Medium	Negative Big	Discard
67	Very High	Medium	Negative	Discard

			Small	
68	Very High	Medium	Zero	Discard
69	Very High	Medium	Positive Small	Strong Discard
70	Very High	Medium	Positive Big	Strong Discard
71	Very High	High	Negative Big	Discard
			Negative	
72	Very High	High	Small	Discard
73	Very High	High	Zero	Strong Discard
74	Very High	High	Positive Small	Strong Discard
75	Very High	High	Positive Big	Strong Discard
		Very		
76	Very High	High	Negative Big	Strong Discard
		Very	Negative	
77	Very High	High	Small	Strong Discard
		Very		
78	Very High	High	Zero	Strong Discard
		Very		
79	Very High	High	Positive Small	Strong Discard
		Very		
80	Very High	High	Positive Big	Strong Discard

APPENDIX B – Data Structures Used

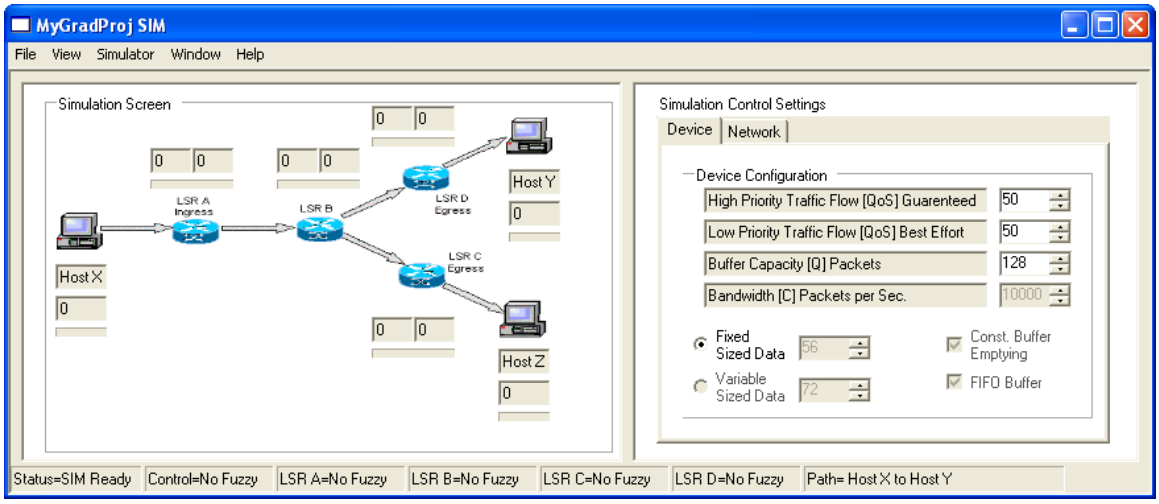
Data Structures Used

```
// Devices with two interfaces
Node structure {
   Node ID                        // Device ID
   In Buffer Pointer to QUEUE     // Inbound buffer
   In Buffer Size (N)             // Inbound buffer queue size
   Out Buffer Pointer to DEQUEUE // Outbound buffer
   Out Buffer Size (N)              // Outbound buffer queue size
   Node Policy In Buffer Keep     // QoS high priority and low priority keep packets
   Node Policy In Buffer Drop     // QoS high priority and low priority drop packets
   Node Interface Path 1          // interface to left device within path
   Node Interface Path 2          // interface to right device within path
}

// Device Inbound buffer
In Buffer QUEUE structure {
   Pointer to QUEUE    // Inbound buffer queue
}

// Device Outbound buffer
Out Buffer DEQUEUE structure {
  Pointer to DEQUEUE // Outbound buffer queue
}

// Network path made up of devices
Path structure {
   Path ID (Path 1 or Path 2)   // Path X -> Y or Path X -> Z only
   Pointer to Node Linked List // Path of devices that packet is routed
}

// Network path made up of devices
Node Linked List structure {
}

// node = device
Node structure {
}

// packet definition
Packet structure {
  Packet ID                 // packet identification
  Priority Flag (high or low) // high priority or low priority packet
  Packet Size (fixed or variable) // a fixed number packets = cell variable number packets = frame
  Pointer to Data Linked List // cell or frame made up of packets
}

// Data Stream/Flow definition
Data Linked List structure {
}

// cell or frame
Data structure {
}
```

APPENDIX C – UI Specification

Windows Section

Simulation with no fuzzy logic controls exposed (Default Window)



Simulation Control Settings

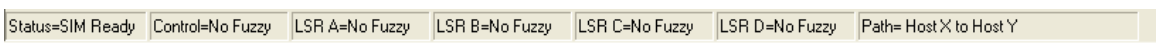
Device Tab

- High Priority Traffic Flow Quality of Service Guaranteed Policy with range between 0 and 100
- Low Priority Traffic Flow Quality of Service Best Efforts Policy with range between 0 and 100
- Buffer Capacity in number of packets with range between 0 and 256
- Bandwidth in number of packets per second with range between 1 and 10000
- Fixed Sized Data in number of packets with range between 0 and 56
- Variable Sized Data in number of packets with range between 0 and 72
- Const Buffer Emptying is set
- First-in-first-out Buffering is set

Network Tab

- Low Traffic Intensity to Congestion as a percentage with range 0 to 100

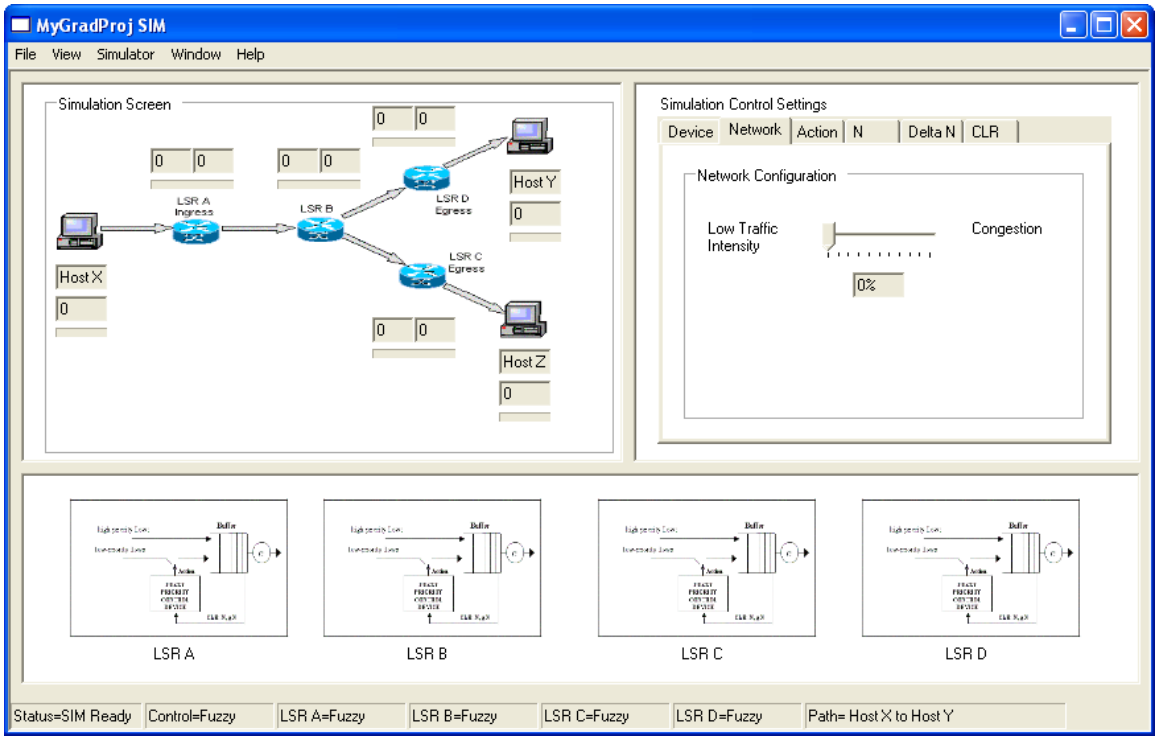
Status



Status = SIM Ready means that the simulator is ready to run
Control = No Fuzzy means that the SIM is not in Fuzzy Logic Mode and no Fuzzy parameters have been applied
LSR A = Fuzzy means that device LSR A is disabled with Fuzzy Logic control scheme

LSR B = Fuzzy means that device LSR A is disabled with Fuzzy Logic control scheme
LSR C = Fuzzy means that device LSR A is disabled with Fuzzy Logic control scheme
LSR D = Fuzzy means that device LSR A is disabled with Fuzzy Logic control scheme
Path = Host X to Host Y means that the path is set between Host X to LSR A to LSR B to LSR D to Host Y

Simulation with fuzzy logic controls exposed



Simulation Control Settings

Action Tab – Output variable made up of size fuzzy sets

- Membership functions for the Action output variable
- X value with range between 0 and 1 for each of the fuzzy sets based on non fuzzy output from membership function
- Y value with range between 0 and 1 for each of the fuzzy sets based on non fuzzy output from membership function

N Tab – Input variable

- Number of packets in buffer at any point in time (N input variable) with range between 0 and 256

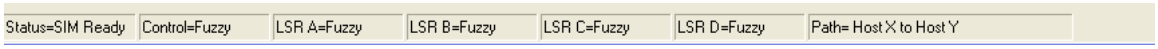
Delta N Tab – Input variable

- Number of packets in buffer over time period (delta N) with range between 0 and 256

CLR Tab – Input variable

- High priority loss ratio in percentage with range between 0 and 100

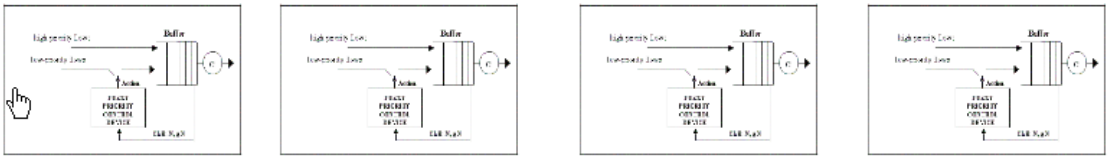
Status



Status = SIM Ready means that the simulator is ready to run
Control = Fuzzy means that the SIM is in Fuzzy Logic Mode with all Fuzzy parameters applied
LSR A = Fuzzy means that device LSR A is enabled with Fuzzy Logic control scheme
LSR B = Fuzzy means that device LSR A is enabled with Fuzzy Logic control scheme

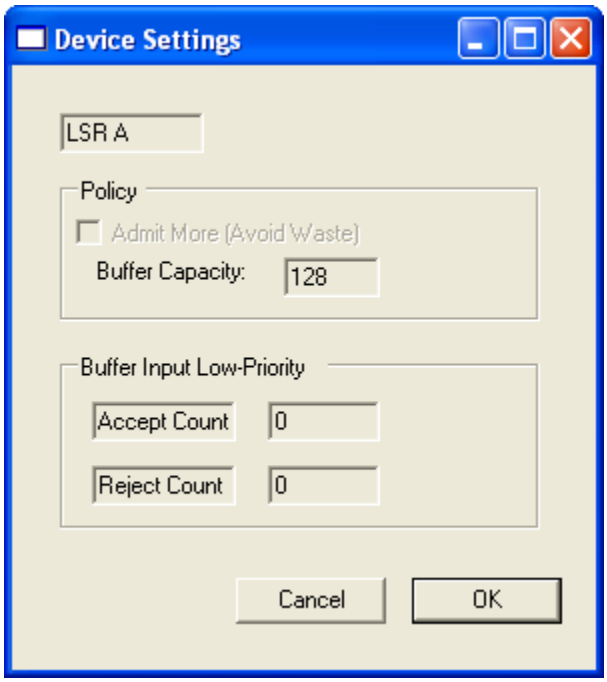
LSR C = Fuzzy means that device LSR A is enabled with Fuzzy Logic control scheme
LSR D = Fuzzy means that device LSR A is enabled with Fuzzy Logic control scheme
Path = Host X to Host Y means that the path is set between Host X to LSR A to LSR B to LSR D to Host Y

Devices



LSR A LSR B LSR C LSR D

LSR A – router device with two interfaces and is the ingress router to the network
LSR B – router device with three interfaces and is the intermediate router within the network
LSR C – router device with two interfaces and is the egress router to the network
LSR D – router device with two interfaces and is the egress router to the network



Device Settings for LSR A, LSR B, LSR C and LSR D

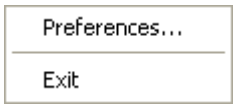
Device Name: LSR A

Device Policy: Admit More Packets to avoid waste
Device Buffer Capacity: 128 packets

Device Buffer Input Low-Priority Packets
Accept Count: 0
Reject Count: 0

Main Menus Section

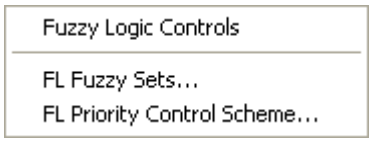
File Menu



Preferences – will display a dialog that allows the user to configure window settings such as setting the output (also called the results) window after each time a simulation run

Exit – allows the user to quit

View Menu

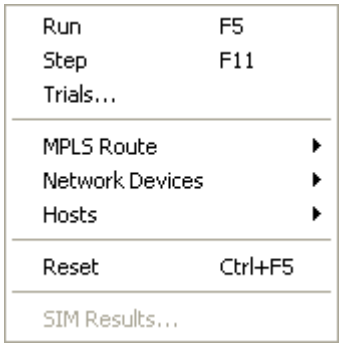


Fuzzy Logic Controls – will display all of the fuzzy controls in the Control region

FL Fuzzy Sets – will display a dialog that contains the six output fuzzy sets

FL Priority Control Scheme – will display a dialog that allows the user to select one or more rules applied out of the possible eighty fuzzy logic rules available when running the simulator in fuzzy logic mode

Simulator Menu

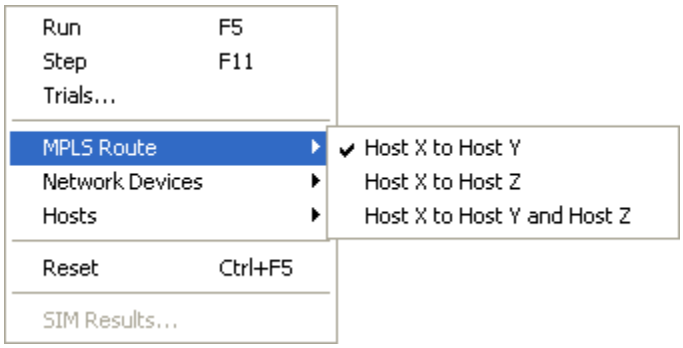


Run – will activate and run the simulation

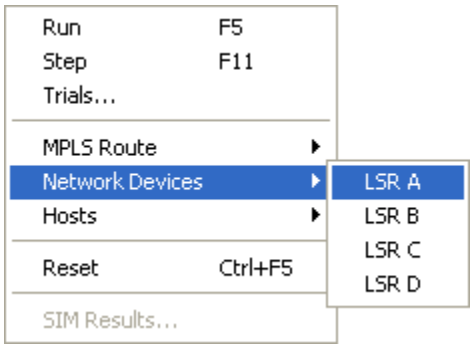
Step – will activate and step through the simulation one step at a time

Trials…

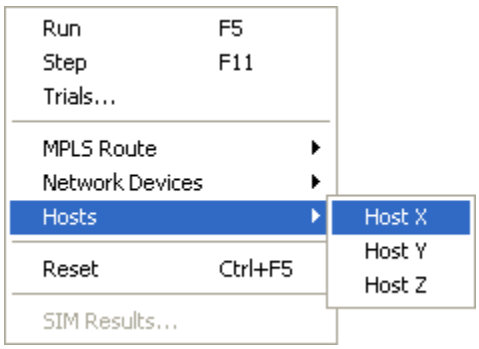
MPLS Route – will display a submenu to allow the user to select a particular path where there are three paths available



Network Devices – will display a submenu to allow the user to select a particular device in order to display device configuration information (note: clicking on the device when the device selection region of the window is visible is also an option).



Hosts – will display a submenu to allow the user to select a particular host in order to display host configuration information.



Reset – will set the simulation back to the beginning

SIM Results… -- by default is disabled but is enabled once a simulation has run at which point a user can select this menu item to display the results of the last simulation

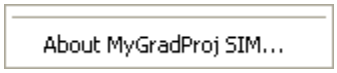
Window Menu

Simulation Screen – will show/hide the primary screen that contains the simulation model

Simulation Controls – will show/hide the secondary screen that contains the simulation control settings

Device Selection – will show/hide all four device mouse roll-overs (or hot regions) that allow the user to quickly access the device settings dialog

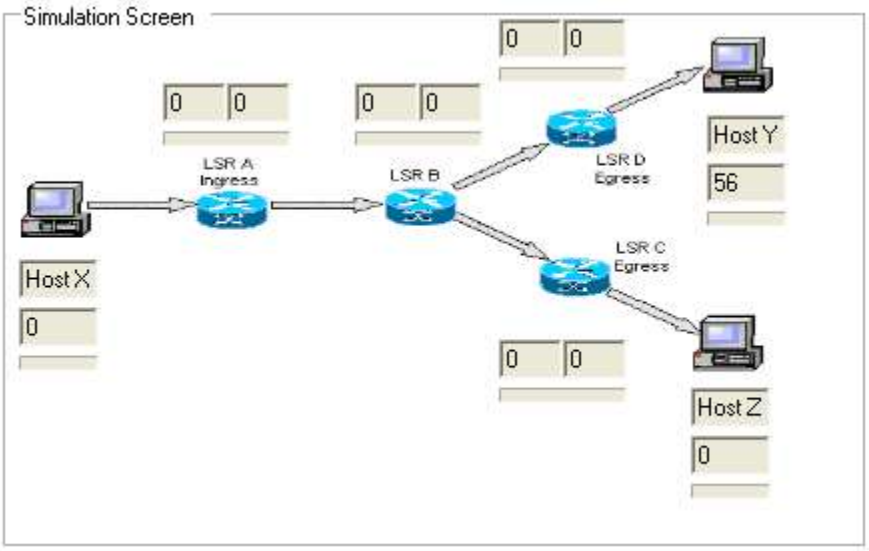
Help Menu



About – will display a dialog containing version reference

Screen Regions

Simulation Region



Simulation Screen Properties

Host X:
Outbound buffer has transferred 0 packets
Progress bar is clear

LSR A: (MPLS Enabled - Label Switch router A)
Inbound Buffer currently has 0 packets
Outbound Buffer currently has 0 packets
Progress bar is clear

LSR B: (MPLS Enabled - Label Switch router B)
Inbound Buffer currently has 0 packets
Outbound Buffer currently has 0 packets
Progress bar is clear

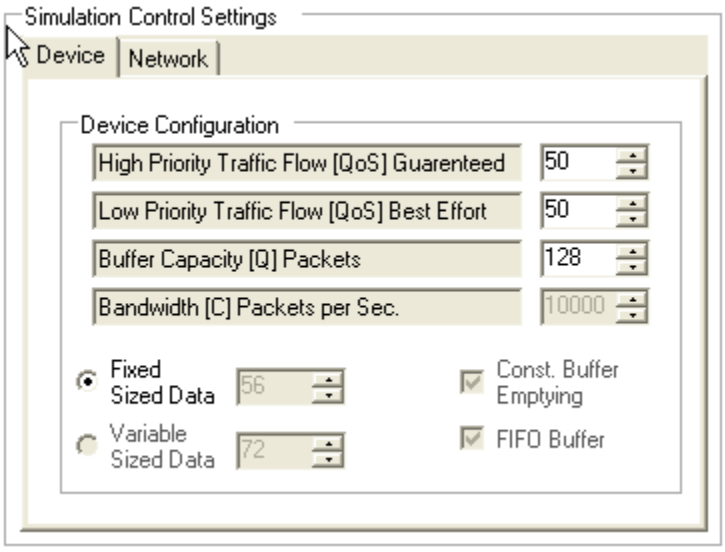
LSR C: (MPLS Enabled - Label Switch router C)
Inbound Buffer currently has 0 packets
Outbound Buffer currently has 0 packets
Progress bar is clear

LSR D: (MPLS Enabled - Label Switch router D)
Inbound Buffer currently has 0 packets
Outbound Buffer currently has 0 packets
Progress bar is clear

Host Y:
Inbound buffer currently has 56 packets
Progress bar is clear

Host Z:
Inbound buffer currently has 0 packets
Progress bar is clear

Controls Region – simulation settings for the model

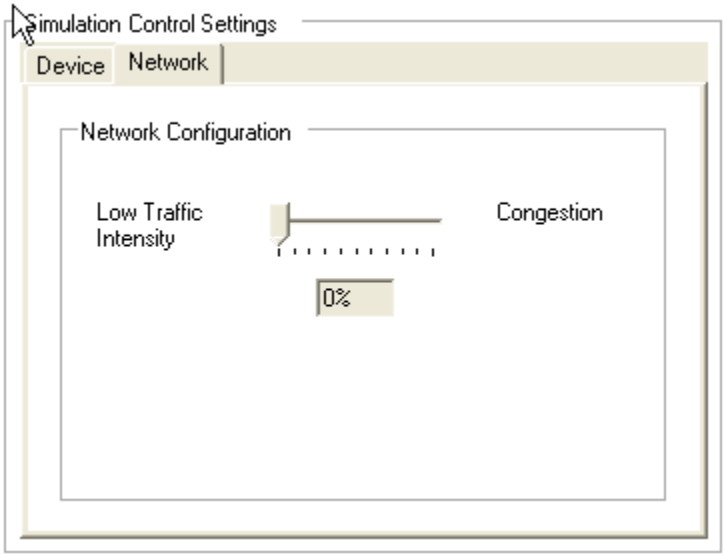


Simulation Control Settings

Device Tab

- High Priority Traffic Flow Quality of Service Guaranteed Policy with range between 0 and 100

- Low Priority Traffic Flow Quality of Service Best Efforts Policy with range between 0 and 100
- Buffer Capacity in number of packets with range between 0 and 256
- Bandwidth in number of packets per second with range between 1 and 10000
- Fixed Sized Data in number of packets with range between 0 and 56
- Variable Sized Data in number of packets with range between 0 and 72
- Const Buffer Emptying is set
- First-in-first-out Buffering is set

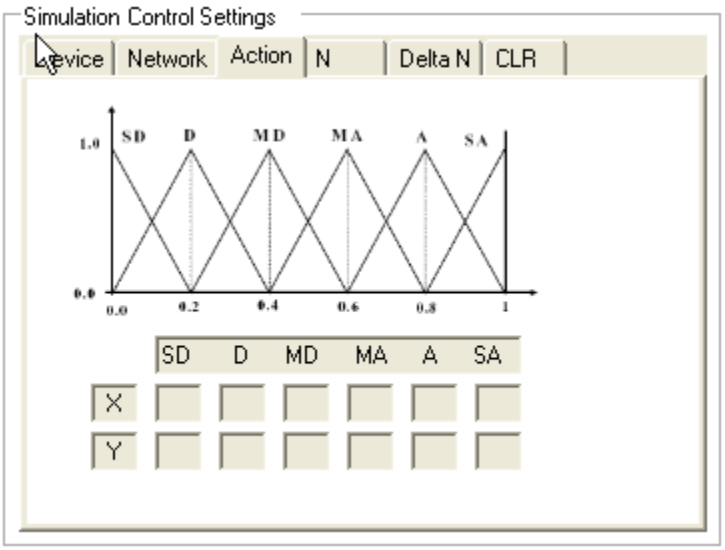


Network Tab

- Low Traffic Intensity to Congestion as a percentage with range 0 to 100

Controls Region – fuzzy logic settings for the model

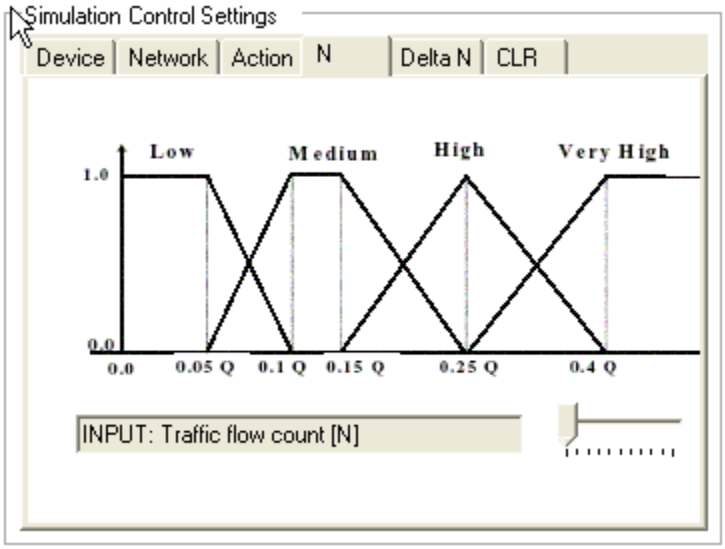
ACTION – fuzzy logic output panel



Action Tab – Output variable made up of size fuzzy sets

- Membership functions for the Action output variable
- SD = Strong Discard, D = Discard, MD = Marginal Discard, MA = Marginal Admit, A = Admit, SA = Strong Admit
- X value with range between 0 and 1 for each of the fuzzy sets based on non fuzzy output from membership function
- Y value with range between 0 and 1 for each of the fuzzy sets based on non fuzzy output from membership function

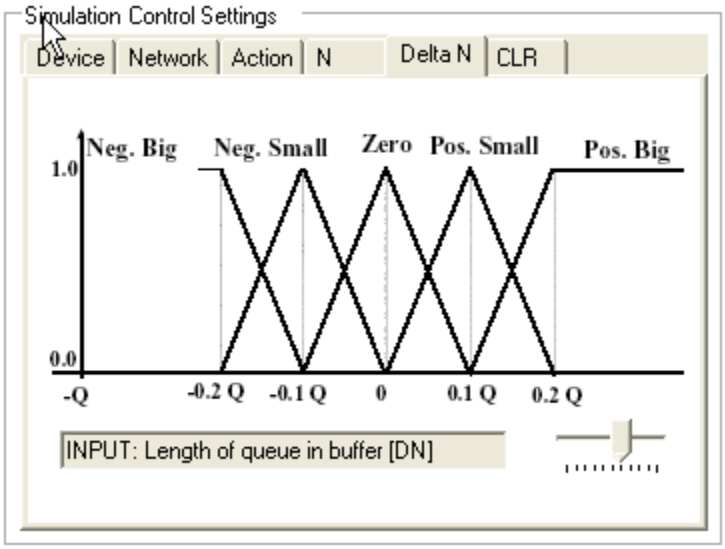
N – fuzzy logic input variable panel



N Tab – Input variable

- Number of packets in buffer at any point in time (N input variable) with range between 0 and 256

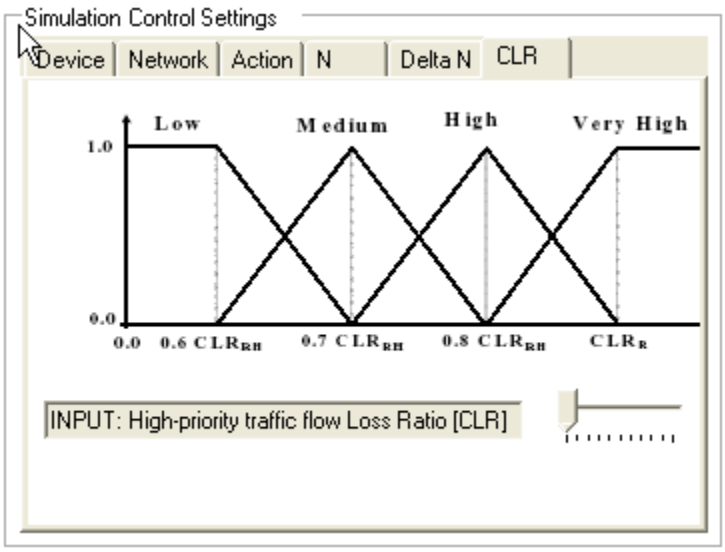
Delta N – fuzzy logic input variable panel



Delta N Tab – Input variable

- Number of packets in buffer over time period (delta N) with range between 0 and 256

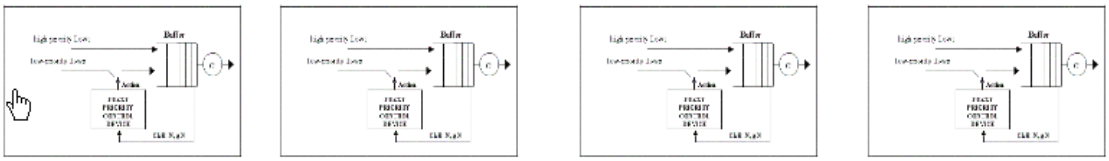
CLR – fuzzy logic input variable panel



CLR Tab – Input variable

- High priority loss ratio in percentage with range between 0 and 100

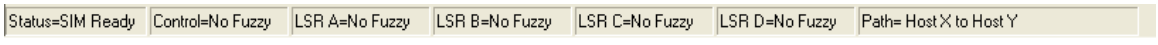
Device Region – four mouse-over regions that allow the user to quickly get device information



LSR A LSR B LSR C LSR D

LSR A – router device with two interfaces and is the ingress router to the network
LSR B – router device with three interfaces and is the intermediate router within the network
LSR C – router device with two interfaces and is the egress router to the network
LSR D – router device with two interfaces and is the egress router to the network

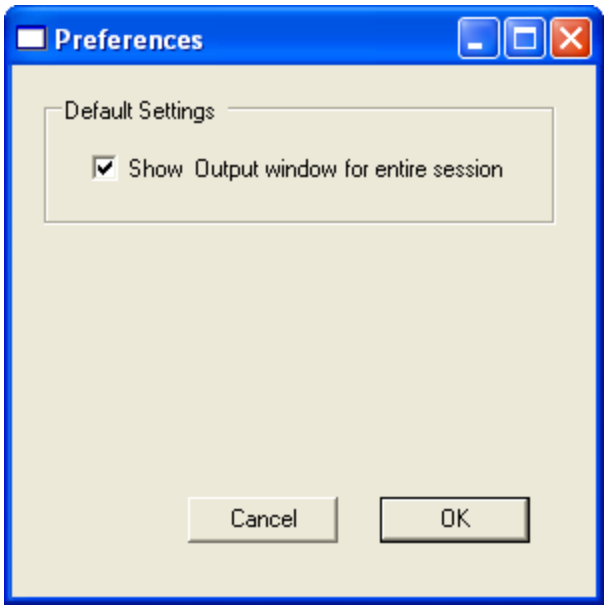
Status Region – status on devices, hosts, controls and errors



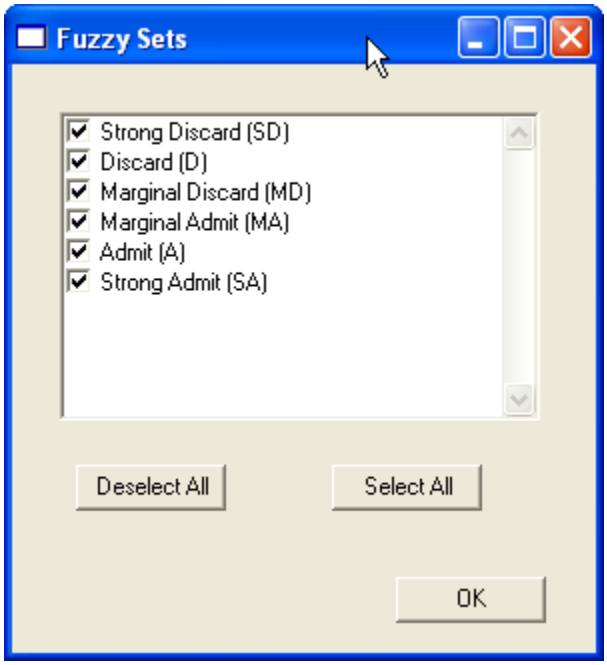
Status = SIM Ready means that the simulator is ready to run
Control = No Fuzzy means that the SIM is not in Fuzzy Logic Mode and no Fuzzy parameters have been applied
LSR A = Fuzzy means that device LSR A is disabled with Fuzzy Logic control scheme
LSR B = Fuzzy means that device LSR A is disabled with Fuzzy Logic control scheme
LSR C = Fuzzy means that device LSR A is disabled with Fuzzy Logic control scheme
LSR D = Fuzzy means that device LSR A is disabled with Fuzzy Logic control scheme
Path = Host X to Host Y means that the path is set between Host X to LSR A to LSR B to LSR D to Host Y

Dialogs

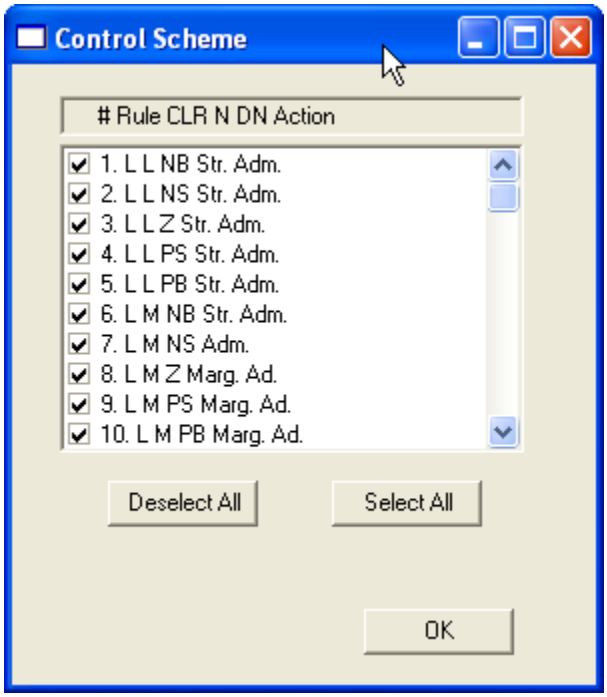
Preferences Dialog – Default Settings to show output (also called results) window after each simulation run



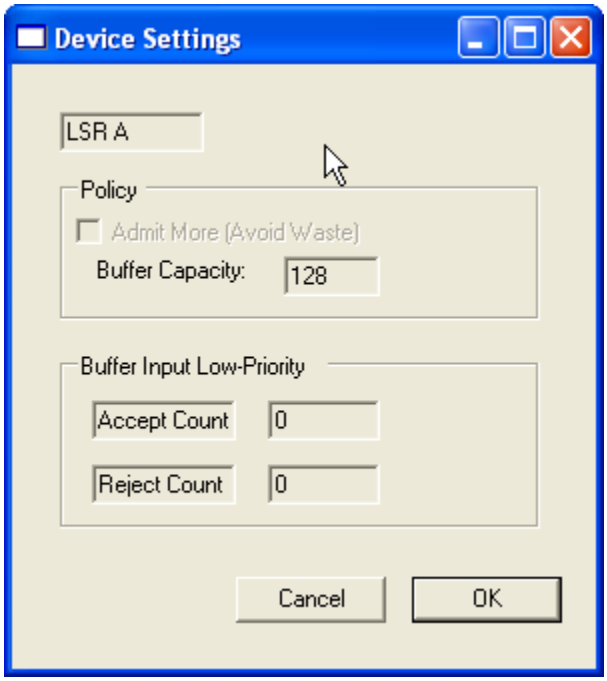
Fuzzy Sets Dialog – six fuzzy sets used in the model



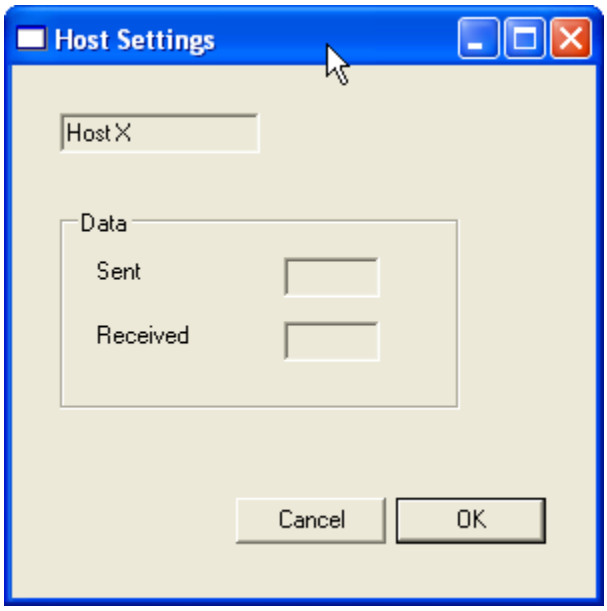
Control Scheme – eighty fuzzy IF-THEN rules used in the model



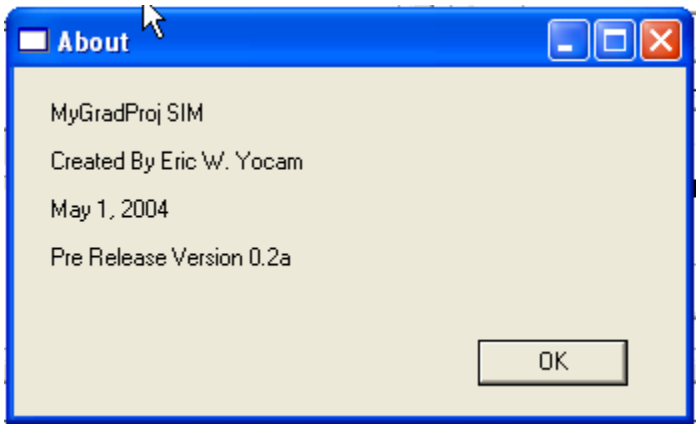
Device Settings – user can change device policy and get read out of buffer input accept/reject rates



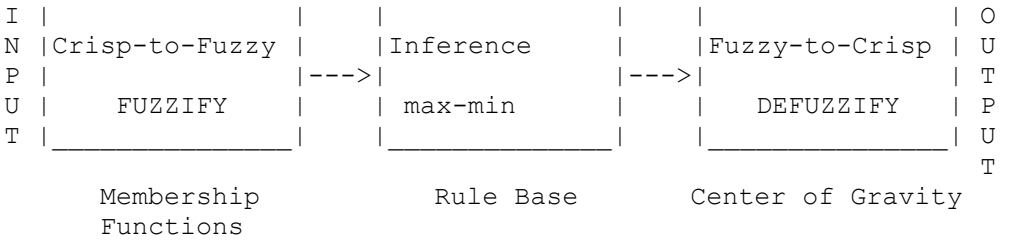
Host Settings – user can get read out of amount of data flow either sent out by the host or received by the host depending on the host selected



About – copyright and version information



APPENDIX D – Max-Min Inference and Defuzzification Equations Based on COG Approach
Adapted from IEC specification document 61131-7 Figure A.2.4-1a: Methods of defuzzification [15]



Center of Gravity (COG) approach is equivalent to Centroid of Area

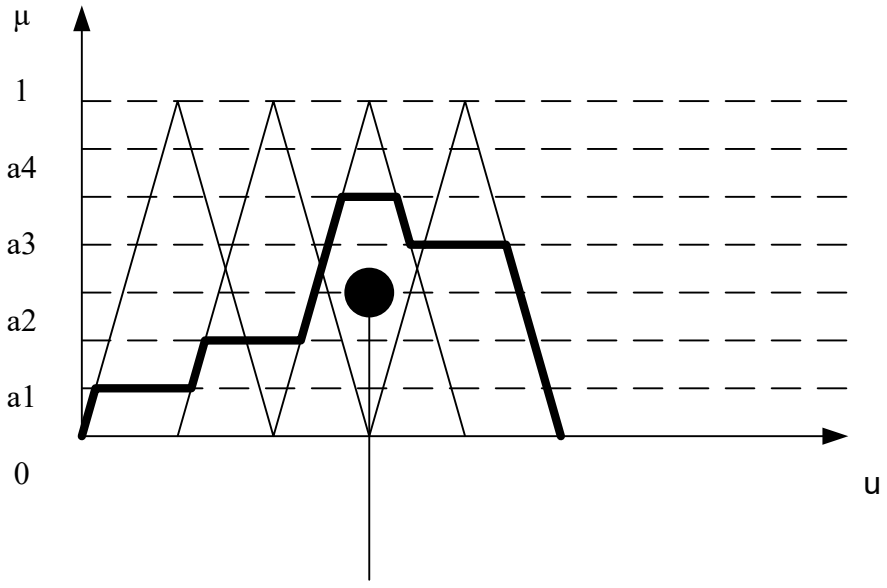
Defuzzification

Conversion of the fuzzy result of inference into a crisp output variable U

Center of Gravity (CoG) Method

The crisp output variable is determined as an abscissa value U of center of gravity under the membership function:

- example for center of gravity method



U : Center of Gravity (COG)

APPENDIX E – Minimum FCL compliance and a listing of production rules (from [20]).

Minimum FCL Compliance

Language Element	Keyword	Details
function block declaration	VAR_INPUT, VAR_OUTPUT	contains input and output variables
membership function	input variable: TERM	maximum of three points (degree of membership co-ordinate = 0 or 1)
	output variable: TERM	only singletons
conditional aggregation	operator: AND	algorithm: MIN
activation		Not relevant because singletons are used only
accumulation (result aggregation)	operator: ACCU	algorithm: MAX
defuzzification	METHOD	algorithm: COGS
condition	*IF ... IS ...*	*n subconditions*
conclusion	*THEN*	*only one subconclusion*
weighting factor	*WITH*	*value only*

Production Rules

```
function_block_declaration ::= 'FUNCTION_BLOCK' function_block_name
                               {fb_io_var_declarations}
                               {other_var_declarations}
                               function_block_body
                           'END_FUNCTION_BLOCK'
fb_io_var_declarations ::= input_declarations | output_declarations
other_var_declarations ::= var_declarations
function_block_body ::= {fuzzify_block}
                        {defuzzify_block}
                        {rule_block}
                        {option_block}
fuzzify_block ::= 'FUZZIFY' variable_name
                        {linguistic_term}
                  'END_FUZZIFY
defuzzify_block ::= 'DEFUZZIFY' f_variable_name
                         {linguistic_term}
                         defuzzification_method
                         default_value
                         [range]
                   'END_FUZZIFY'
rule_block ::= 'RULEBLOCK' rule_block_name
                        operator_definition
                        [activation_method]
                        accumulation_method
                        {rule}
               'END_RULEBLOCK'
option_block ::= 'OPTION'
                         any manufacturere specific parameter
                 'END_OPTION'
linguistic_term ::= 'TERM' term_name ':=' membership_function ';'
```

```
membership_function ::= singleton | points
singleton ::= numeric_literal | variable_name
points ::= {‘(‘ numeric_literal | variable_name ‘,’ numeric_literal ‘)’}
defuzzification_method ::= ‘METHOD’ ‘:’ ‘COG’ | ‘COGS’ | ‘COA’ | ‘LM’ | ‘RM’ | MOM‘;’
default_value ::= ‘DEFAULT’ ‘:=‘ numeric_literal | ‘NC’ ‘;’
range ::= ‘RANGE’ ‘:=‘ ‘(‘numeric_literal ‘..’ numeric_literal‘)’ ‘;’
operator_definition ::= (‘OR’ ‘:’ ‘MAX’ | ‘ASUM’ | ‘BSUM’) | (‘AND’ ‘:’ ‘MIN’ | ‘PROD’ | ‘BDIF’) ‘;’
activation_method ::= ‘ACT’ ‘:’ ‘PROD’ | ‘MIN’ ‘;’
accumulation_method ::= ‘ACCU’ ‘:’ ‘MAX’ | ‘BSUM’ | ‘NSUM’ ‘;’
rule ::= ‘RULE’ integer_literal ‘:’
        ‘IF’ condition ‘THEN’ conclusion [WITH weighting_factor] ‘;’
condition ::= (subcondition | variable_name) {AND’ | ‘OR’ (subcondition | variable_name)}
subcondition ::= (‘NOT’ ‘(‘ variable_name ‘IS’ [‘NOT’] ) term_name ‘)’) | ( variable_name ‘IS’ [‘NOT’] term_name )
FFLL shorthand: subcondition ::= term_name

conclusion ::= { (variable_name | (variable_name ‘IS’ term_name)) ‘,’}
weighting_factor ::= variable | numeric_literal
function_block_name ::= identifier
rule_block_name ::= identifier
term_name ::= identifier
variable_name ::= identifier
numeric_literal ::= integer_literal | real_literal
letter ::= 'A' | 'B' | <...> | 'Z' | 'a' | 'b' | <...> | 'z'
digit ::= '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'
identifier ::= (letter | ('_' (letter | digit))) {['_'] (letter | digit)}
input_declarations ::=
'VAR_INPUT' ['RETAIN' | 'NON_RETAIN']
input_declaration ';'
{input_declaration ';'}
'END_VAR'
input_declaration ::= var_init_decl | edge_declaration

var_init_decl ::= var1_init_decl | array_var_init_decl | structured_var_init_decl | fb_name_decl |
string_var_declaration

var1_init_decl ::= var1_list ':'
(simple_spec_init | subrange_spec_init | enumerated_spec_init)
var1_list ::= variable_name {',' variable_name}
array_var_init_decl ::= var1_list ':' array_spec_init
output_declarations ::=
'VAR_OUTPUT' ['RETAIN' | 'NON_RETAIN']
var_init_decl ';'
{var_init_decl ';'}
'END_VAR'

real_type_name ::= 'REAL' | 'LREAL'
numeric_type_name ::= integer_type_name | real_type_name
elementary_type_name ::= numeric_type_name | date_type_name | bit_string_type_name | 'STRING' |
'WSTRING' | 'TIME'
simple_type_name ::= identifier
simple_type_declaration ::= simple_type_name ':' simple_spec_init
simple_specification ::= elementary_type_name | simple_type_name
simple_spec_init := simple_specification [':=' constant]
```

APPENDIX F – FCL file used in the simulator

```
(* FCL File Created From FFLL Model for MyGradProj *)
FUNCTION_BLOCK

VAR_INPUT
	N_IN		REAL; (* RANGE(0 .. 100) *)
	DN_IN		REAL; (* RANGE(0 .. 100) *)
	CLR_IN		REAL; (* RANGE(0 .. 100) *)
END_VAR

VAR_OUTPUT
	ACTION		REAL; (* RANGE(0 .. 7) *)
END_VAR

FUZZIFY N_IN
	TERM Low := (0, 0) (0, 1) (50, 0) ;
	TERM Medium := (14, 0) (50, 1) (83, 0) ;
	TERM High := (50, 0) (100, 1) (100, 0) ;
	TERM Very_High := (50, 0) (100, 1) (100, 0) ;
END_FUZZIFY

FUZZIFY DN_IN
	TERM Negative_Big := (0, 0) (0, 1) (50, 0) ;
	TERM Negative_Small := (14, 0) (50, 1) (83, 0) ;
	TERM Zero := (50, 0) (100, 1) (100, 0) ;
	TERM Positive_Small := (50, 0) (100, 1) (100, 0) ;
	TERM Positive_Big := (50, 0) (100, 1) (100, 0) ;
END_FUZZIFY

FUZZIFY CLR_IN
	TERM Low := (0, 0) (0, 1) (50, 0) ;
	TERM Medium := (14, 0) (50, 1) (80, 0) ;
	TERM High := (50, 0) (100, 1) (100, 0) ;
	TERM Very_High := (50, 0) (100, 1) (100, 0) ;
END_FUZZIFY

FUZZIFY ACTION
	TERM Strong_Discard :=  1;
	TERM Discard :=  2;
	TERM Marginal_Discard :=  3;
	TERM Marginal_Admit :=  4;
	TERM Admit :=  5;
	TERM Strong_Admit :=  6;
END_FUZZIFY

DEFUZZIFY ACTION
	METHOD: CoG;
END_DEFUZZIFY

RULEBLOCK first
	AND:MIN;
	ACCUM:MAX;
	RULE 0: IF (CLR IS Low) AND (N IS Low) AND (DN IS Negative_Big) THEN (ACTION IS Strong_Admit);
	RULE 1: IF (CLR IS Low) AND (N IS Low) AND (DN IS Negative_Small) THEN (ACTION IS Strong_Admit);
	RULE 2: IF (CLR IS Low) AND (N IS Low) AND (DN IS Zero) THEN (ACTION IS Strong_Admit);
	RULE 3: IF (CLR IS Low) AND (N IS Low) AND (DN IS Positive_Small) THEN (ACTION IS Strong_Admit);
	RULE 4: IF (CLR IS Low) AND (N IS Low) AND (DN IS Positive_Big) THEN (ACTION IS Strong_Admit);
	RULE 5: IF (CLR IS Low) AND (N IS Medium) AND (DN IS Negative_Big) THEN (ACTION IS Strong_Admit);
	RULE 6: IF (CLR IS Low) AND (N IS Medium) AND (DN IS Negative_Small) THEN (ACTION IS Admit);
	RULE 7: IF (CLR IS Low) AND (N IS Medium) AND (DN IS Zero) THEN (ACTION IS Marginal_Admit);
	RULE 8: IF (CLR IS Low) AND (N IS Medium) AND (DN IS Positive_Small) THEN (ACTION IS
Marginal_Admit);
	RULE 9: IF (CLR IS Low) AND (N IS Medium) AND (DN IS Positive_Big) THEN (ACTION IS Marginal_Admit);
	RULE 10: IF (CLR IS Low) AND (N IS High) AND (DN IS Negative_Big) THEN (ACTION IS Admit);
	RULE 11: IF (CLR IS Low) AND (N IS High) AND (DN IS Negative_Small) THEN (ACTION IS Marginal_Admit);
	RULE 12: IF (CLR IS Low) AND (N IS High) AND (DN IS Zero) THEN (ACTION IS Marginal_Admit);
	RULE 13: IF (CLR IS Low) AND (N IS High) AND (DN IS Positive_Small) THEN (ACTION IS
Marginal_Discard);
```

RULE 14: IF (CLR IS Low) AND (N IS High) AND (DN IS Positive_Big) THEN (ACTION IS Marginal_Discard);
RULE 15: IF (CLR IS Low) AND (N IS Very_High) AND (DN IS Negative_Big) THEN (ACTION IS Marginal_Discard);
RULE 16: IF (CLR IS Low) AND (N IS Very_High) AND (DN IS Negative_Small) THEN (ACTION IS Marginal_Discard);
RULE 17: IF (CLR IS Low) AND (N IS Very_High) AND (DN IS Zero) THEN (ACTION IS Marginal_Discard);
RULE 18: IF (CLR IS Low) AND (N IS Very_High) AND (DN IS Positive_Small) THEN (ACTION IS Discard);
RULE 19: IF (CLR IS Low) AND (N IS Very_High) AND (DN IS Positive_Big) THEN (ACTION IS Discard);
RULE 20: IF (CLR IS Medium) AND (N IS Low) AND (DN IS Negative_Big) THEN (ACTION IS Strong_Admit);
RULE 21: IF (CLR IS Medium) AND (N IS Low) AND (DN IS Negative_Small) THEN (ACTION IS Admit);
RULE 22: IF (CLR IS Medium) AND (N IS Low) AND (DN IS Zero) THEN (ACTION IS Admit);
RULE 23: IF (CLR IS Medium) AND (N IS Low) AND (DN IS Positive_Small) THEN (ACTION IS Admit);
RULE 24: IF (CLR IS Medium) AND (N IS Low) AND (DN IS Positive_Big) THEN (ACTION IS Marginal_Admit);
RULE 25: IF (CLR IS Medium) AND (N IS Medium) AND (DN IS Negative_Big) THEN (ACTION IS Admit);
RULE 26: IF (CLR IS Medium) AND (N IS Medium) AND (DN IS Negative_Small) THEN (ACTION IS Admit);
RULE 27: IF (CLR IS Medium) AND (N IS Medium) AND (DN IS Zero) THEN (ACTION IS Marginal_Admit);
RULE 28: IF (CLR IS Medium) AND (N IS Medium) AND (DN IS Positive_Small) THEN (ACTION IS Marginal_Admit);
RULE 29: IF (CLR IS Medium) AND (N IS Medium) AND (DN IS Positive_Big) THEN (ACTION IS Marginal_Admit);
RULE 30: IF (CLR IS Medium) AND (N IS High) AND (DN IS Negative_Big) THEN (ACTION IS Marginal_Admit);
RULE 31: IF (CLR IS Medium) AND (N IS High) AND (DN IS Negative_Small) THEN (ACTION IS Marginal_Admit);
RULE 32: IF (CLR IS Medium) AND (N IS High) AND (DN IS Zero) THEN (ACTION IS Marginal_Discard);
RULE 33: IF (CLR IS Medium) AND (N IS High) AND (DN IS Positive_Small) THEN (ACTION IS Marginal_Discard);
RULE 34: IF (CLR IS Medium) AND (N IS High) AND (DN IS Positive_Big) THEN (ACTION IS Marginal_Discard);
RULE 35: IF (CLR IS Medium) AND (N IS Very_High) AND (DN IS Negative_Big) THEN (ACTION IS Marginal_Discard);
RULE 36: IF (CLR IS Medium) AND (N IS Very_High) AND (DN IS Negative_Small) THEN (ACTION IS Marginal_Discard);
RULE 37: IF (CLR IS Medium) AND (N IS Very_High) AND (DN IS Zero) THEN (ACTION IS Discard);
RULE 38: IF (CLR IS Medium) AND (N IS Very_High) AND (DN IS Positive_Small) THEN (ACTION IS Discard);
RULE 39: IF (CLR IS Medium) AND (N IS Very_High) AND (DN IS Positive_Big) THEN (ACTION IS Strong_Discard);
RULE 40: IF (CLR IS High) AND (N IS Low) AND (DN IS Negative_Big) THEN (ACTION IS Admit);
RULE 41: IF (CLR IS High) AND (N IS Low) AND (DN IS Negative_Small) THEN (ACTION IS Admit);
RULE 42: IF (CLR IS High) AND (N IS Low) AND (DN IS Zero) THEN (ACTION IS Marginal_Admit);
RULE 43: IF (CLR IS High) AND (N IS Low) AND (DN IS Positive_Small) THEN (ACTION IS Marginal_Admit);
RULE 44: IF (CLR IS High) AND (N IS Low) AND (DN IS Positive_Big) THEN (ACTION IS Marginal_Admit);
RULE 45: IF (CLR IS High) AND (N IS Medium) AND (DN IS Negative_Big) THEN (ACTION IS Marginal_Admit);
RULE 46: IF (CLR IS High) AND (N IS Medium) AND (DN IS Negative_Small) THEN (ACTION IS Marginal_Admit);
RULE 47: IF (CLR IS High) AND (N IS Medium) AND (DN IS Zero) THEN (ACTION IS Marginal_Admit);
RULE 48: IF (CLR IS High) AND (N IS Medium) AND (DN IS Positive_Small) THEN (ACTION IS Marginal_Discard);
RULE 49: IF (CLR IS High) AND (N IS Medium) AND (DN IS Positive_Big) THEN (ACTION IS Discard);
RULE 50: IF (CLR IS High) AND (N IS High) AND (DN IS Negative_Big) THEN (ACTION IS Marginal_Admit);
RULE 51: IF (CLR IS High) AND (N IS High) AND (DN IS Negative_Small) THEN (ACTION IS Marginal_Discard);
RULE 52: IF (CLR IS High) AND (N IS High) AND (DN IS Zero) THEN (ACTION IS Discard);
RULE 53: IF (CLR IS High) AND (N IS High) AND (DN IS Positive_Small) THEN (ACTION IS Discard);
RULE 54: IF (CLR IS High) AND (N IS High) AND (DN IS Positive_Big) THEN (ACTION IS Discard);
RULE 55: IF (CLR IS High) AND (N IS Very_High) AND (DN IS Negative_Big) THEN (ACTION IS Marginal_Discard);
RULE 56: IF (CLR IS High) AND (N IS Very_High) AND (DN IS Negative_Small) THEN (ACTION IS Discard);
RULE 57: IF (CLR IS High) AND (N IS Very_High) AND (DN IS Zero) THEN (ACTION IS Discard);
RULE 58: IF (CLR IS High) AND (N IS Very_High) AND (DN IS Positive_Small) THEN (ACTION IS Discard);
RULE 59: IF (CLR IS High) AND (N IS Very_High) AND (DN IS Positive_Big) THEN (ACTION IS Strong_Discard);
RULE 60: IF (CLR IS Very_High) AND (N IS Low) AND (DN IS Negative_Big) THEN (ACTION IS Marginal_Admit);
RULE 61: IF (CLR IS Very_High) AND (N IS Low) AND (DN IS Negative_Small) THEN (ACTION IS Marginal_Admit);
RULE 62: IF (CLR IS Very_High) AND (N IS Low) AND (DN IS Zero) THEN (ACTION IS Marginal_Admit);
RULE 63: IF (CLR IS Very_High) AND (N IS Low) AND (DN IS Postivie_Small) THEN (ACTION IS Marginal_Admit);

```
        RULE 64: IF (CLR IS Very_High) AND (N IS Low) AND (DN IS Postivie_Big) THEN (ACTION IS
Marginal_Admit);
        RULE 65: IF (CLR IS Very_High) AND (N IS Medium) AND (DN IS Negative_Big) THEN (ACTION IS Discard);
        RULE 66: IF (CLR IS Very_High) AND (N IS Medium) AND (DN IS Negative_Small) THEN (ACTION IS
Discard);
        RULE 67: IF (CLR IS Very_High) AND (N IS Medium) AND (DN IS Zero) THEN (ACTION IS Discard);
        RULE 68: IF (CLR IS Very_High) AND (N IS Medium) AND (DN IS Positive_Small) THEN (ACTION IS
Strong_Discard);
        RULE 69: IF (CLR IS Very_High) AND (N IS Medium) AND (DN IS Positive_Big) THEN (ACTION IS
Strong_Discard);
        RULE 70: IF (CLR IS Very_High) AND (N IS High) AND (DN IS Negative_Big) THEN (ACTION IS Discard);
        RULE 71: IF (CLR IS Very_High) AND (N IS High) AND (DN IS Negative_Small) THEN (ACTION IS Discard);
        RULE 72: IF (CLR IS Very_High) AND (N IS High) AND (DN IS Zero) THEN (ACTION IS Strong_Discard);
        RULE 73: IF (CLR IS Very_High) AND (N IS High) AND (DN IS Positive_Small) THEN (ACTION IS
Strong_Discard);
        RULE 74: IF (CLR IS Very_High) AND (N IS High) AND (DN IS Positive_Big) THEN (ACTION IS
Strong_Discard);
        RULE 75: IF (CLR IS Very_High) AND (N IS Very_High) AND (DN IS Negative_Big) THEN (ACTION IS
Strong_Discard);
        RULE 76: IF (CLR IS Very_High) AND (N IS Very_High) AND (DN IS Negative_Small) THEN (ACTION IS
Strong_Discard);
        RULE 77: IF (CLR IS Very_High) AND (N IS Very_High) AND (DN IS Zero) THEN (ACTION IS
Strong_Discard);
        RULE 78: IF (CLR IS Very_High) AND (N IS Very_High) AND (DN IS Positive_Small) THEN (ACTION IS
Strong_Discard);
        RULE 79: IF (CLR IS Very_High) AND (N IS Very_High) AND (DN IS Positive_Big) THEN (ACTION IS
Strong_Discard);
END_RULEBLOCK

END_FUNCTION_BLOCK
```